

Conference - Meeting Cpp 2025 - Trip report

Event	Meeting C++ Conference, 2025
Author	Guillaume Dua
Date	11/06/2025 - 11/08/2025
My attendance sponsor	Fairmat
Revision	R1



Table of content

- [Conference - Meeting Cpp 2025 - Trip report](#)
 - [Table of content](#)
 - [Introduction](#)
 -  [11/06/2025](#)
 -  **Jens Weller**: Welcome Message
 - Sponsors
 - Creating a **safe** and **inclusive** environment
 - In memory of Rainer Grimm
 -  **Anthony Williams**: Software and safety (Opening Keynote)
 - What safety means ?
 - Standards and mindset
 - Why systems fail ?
 - Undefined behaviors (UB) and reliability
 - Contracts in C++26 as a safety leverage
 - Defense in depth, using appropriate strategy and tools
 - The Arene Base library
 - My takeaway
 -  **Sebastian Theophil**: To Err is Human: Robust Error Handling in C++26
 - The landscape of error handling: historical perspective
 - The modern stuffs: `std::expected`
 - The modern stuffs: contracts
 - Library hardening in C++26
 - Practical rules for real-world code

- About **ThinkCell**'s error reporting
 - My takeaway
 - Additional resources: learn more about error handling
-  **Nicolai Josuttis**: Using `std::generator<>` in Practice
 - What coroutines are (and aren't)
 - What `std::generator` brings
 - Pitfalls and caveats
 - My takeaway
 - Additional resources: learn more about coroutines
-  **Sandor Dargo**: Code Reviews: Building Better Code and Stronger Teams
 - The value of code reviews
 - Common arguments and misconceptions
 - The human aspects
 - Sandor's **AIR** methodology
 - My takeaway
 - Bonus: How I personally do code reviews
 - Conclusion
-  **Anders Schau Knatten** The Two Memory Models
 - What are the two memory models ?
 - At language-level
 - At CPU-level
 - Mental model for developers
 - My takeaway
 - Additional resources
-  11/07/2025
 -  **Jens Weller** - Good Morning Meeting C++
 - My takeaway
 -  **Frances Buontempo** - Center Keynote - Stoopid Questions
 - My takeaway
 -  **Jonathan Müller** - Cache-Friendly C++
 - Cache-friendly code: why does it matter ?
 - Data structures matter
 - Type size and layout affect cache use
 - Prefetching & cache lines
 - Code is data too
 - Data-Oriented Design (DOD)
 - Multicore pitfalls
 - My takeaway
 -  **Andreas Fertig** - Embedded-Friendly C++: Features That Make a Difference
 - Content
 - My takeaway
 -  **Daniela Engert** - Towards Safety and Security in C++26
 - Content
 - My takeaway
 -  **Raymi Klingers** - 25+ Years of pathfinding problems with C++
 - Content

- My takeaway
-  11/08/2025
 -  **Klaus Iglberger** - The Real Problem of C++
 - Safety vs. performance: an historical tradeoff
 - Bounds safety
 - Undefined behaviors: the silent killers
 - Type safety through strong types
 - Lifetime safety: value semantics over inheritance
 - Conclusion: the real problem is a **people problem**
 - My takeaway
 - Additional sources
 -  **Roth Michaels** - How to become obsolete: a guide to software engineering mentorship
 - Why become obsolete ?
 - Shared technical leadership
 - Identifying and developing leaders
 - Practical mentorship techniques
 - Sharing knowledge
 - Talking through engineering problems, asking them
 - Pair-debugging is an incredibly powerful teaching tool
 - Code-reviews: differentiate types of comments
 - Encourage psychological safety
 - Q&A
 - My Takeaway
 -  **Kerstin Keller** - Why managing C++ dependencies is hard (and what to do about it)
 - What are dependencies & what is **dependency management** ?
 - How developers handle dependencies nowadays ?
 - Why C++ dependencies management is unique and hard ?
 - Inputs from the Q&A
 - My Takeaway
 - Additional resources
 -  **Jens Weller** - Meeting C++ update
 - What's next ?
 - 2026 conference
 - Hommage to Rainer Grimm
 - ⚡ Lightning talks
 - ⚡ **James McNellis** Our most teacherous adversary
 - ⚡ **Rahel Natalie Engel** - let them eat cake
 - ⚡ **Robin Savenen Soderholm** - vector to array: let the compiler turn heap into static storage
 - ⚡ **Ganest Rengasamy** - Meet Qt
 - ⚡ **Hannah Lenk** - Start teaching C++
 - ⚡ **Jens Weller** - binary trees
 - My takeaway
 -  **James McNellis** - Closing Keynote - A little introduction to control flow integrity
 - My takeaway

- [👉 Allow me a digression here](#)
 - [Let's code](#)
 - [So what is the takeaway for me on this ?](#)
- [Conclusion](#)
- [About](#)
- [Related posts](#)

Introduction

This year - and like every year: see my [trip-report for last year's conference](#) - I attended Meeting C++ 2025. As always, the conference brought together a vibrant and passionate community of C++ developers, with a **strong emphasis this year on safety**, security, performance, as well as the human side of software development: particularly the role of lead developers, mentorship, and empathetic collaboration.

While I joined from home, I still very much enjoyed the experience of participating in discussions, chatting with attendees online, and learning from an excellent lineup of talks.

Also, I would like to sincerely thank my employer - [Fairmat](#) - for funding my participation in the conference for the second year in a row.

⚠ Disclaimer:

The opinions, interpretations, and conclusions expressed in this report - *and the "my takeaway", "digression", and "conclusion" sections in particular* - are solely my own and do not necessarily reflect the views of my employer.

While my participation in the conference was funded by my employer, they bear no responsibility for the content, opinions, or interpretations presented here.

From this - admittedly far too long - trip report, I hope that if you couldn't attend the conference this year, you can at least take away a small piece of the experience. And who knows, if you enjoy reading it, perhaps you'll join in person or remotely next year 😊.

? Why spend nearly three months writing a >80 commits, almost 2000-lines trip report ?

First and foremost, this document is for future me. I cannot `ctrl+f` my own memory, but I can search this report to quickly recover who said what, about which topic, and in what context.

Like any conference report, it cannot fully capture the depth, precision, and nuance of every talk; what follows is therefore my own understanding and interpretation of the speakers' messages.

Beyond documentation, the act of writing itself matters: it helps me internalize the ideas, connect themes across talks, and turn fleeting impressions into lasting knowledge.

▶▶ What's coming next ?

I'll make sure to add [📺 videos](#) and [📄 slides](#) links when available.

Resources:

- [Schedule](#)
- [Slides](#)
- [Youtube Channel](#)

 11/06/2025

Jens Weller: Welcome Message

The conference opened, as tradition, with a warm welcome from Jens Weller, the founder and organizer of Meeting C++.

Jens provided an overview of what to expect during the event:

- 3 keynotes
- 54 talks spread across 4 tracks (plus an additional online-exclusive track)
- About 230 attendees onsite and 150 remote attendees

Sponsors

- Gold sponsors: Think-cell, KDAB, Bloomberg
- Video sponsors: [Woven by Toyota](#), JetBrains
- T-shirt sponsor: HRT
- Online sponsor: Qt

Creating a safe and inclusive environment

Jens reiterated the importance of a welcoming community, referencing the [Berlin code of conduct](#). A dedicated Trust & Safety Team was available to ensure every participant could enjoy the conference comfortably.

He also acknowledged the continued attention to health considerations, including SARS-CoV2 prevention policies.

In memory of Rainer Grimm

This year's introduction carried a deeply emotional moment, with [the remembrance of Rainer Grimm](#), who passed away on October 6th 2025, from ALS.

Rainer has been a beloved figure in the C++ community - through his blog, his teaching, his books, and his presence at many conferences.

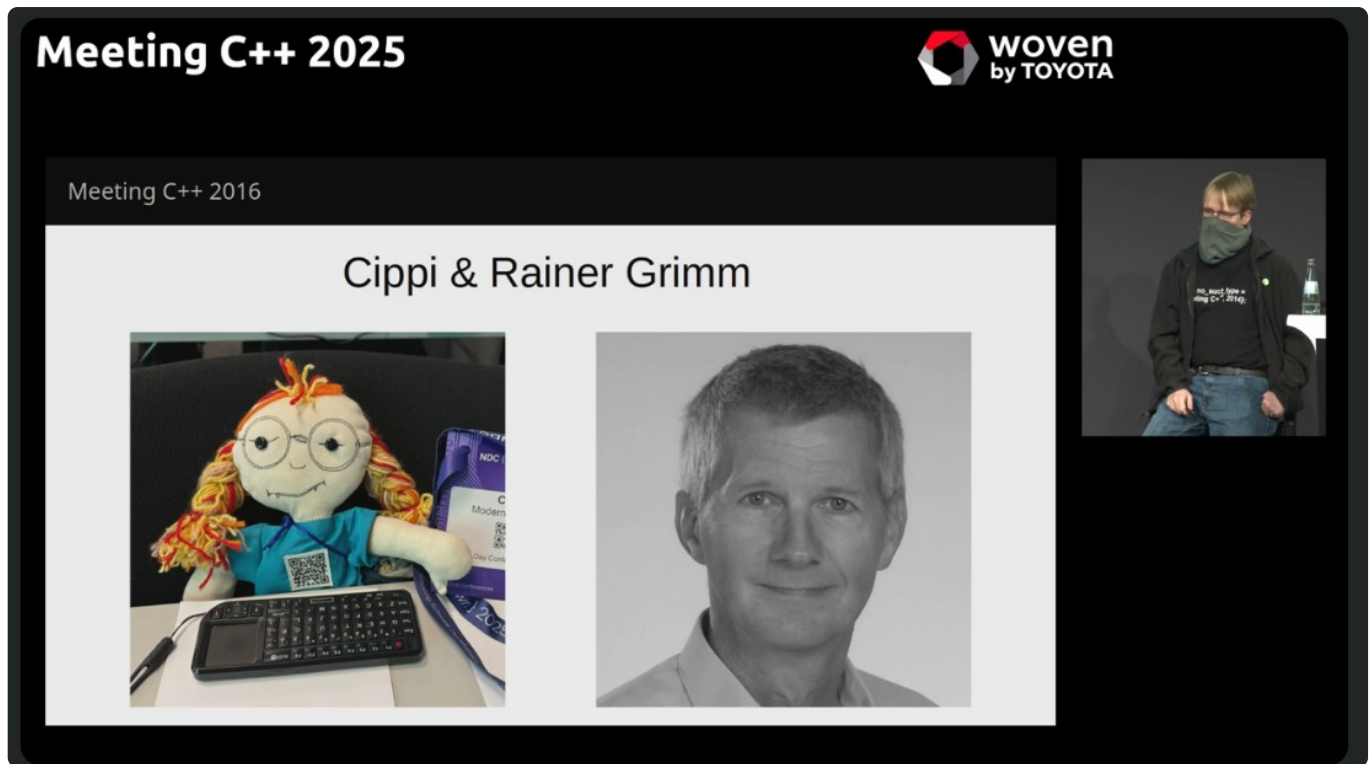
A small plush mascot named [Cippi](#) was on stage next to Jens, as Rainer's symbolic presence.

Instead of the planned session for this year, the conference replayed his talk from last year: an overview of upcoming C++26 features.

Attendees were encouraged to share messages for Rainer's family and community via a QR code linking to his website, on Cippi's tshirt.

[Klaus Iglberger](#) (one of my favorite conference speaker) will continue carrying Cippi at future conferences, preserving Rainer's memory and acknowledging his impact.

This tribute was a heartfelt reminder of the human connections that bind the C++ community together. ❤️



 **Anthony Williams:** Software and safety (Opening Keynote)

 Video,  slidedeck

[Anthony Williams](#) - ISO C++ Committee member (contracts) and low-level library developer at [Woven by Toyota](#) - opened the conference with "Software and Safety", a talk about how principles from safety-critical domains can (and should !) influence **all** software developments.

What safety means ?

Safety-critical software is any system where failure can cause injury or loss of life: automotive control, aircraft systems, factory and nuclear controls, or medical devices like pacemakers and CT scanners.

But even outside those domains, **software failures can cause real harm**: financial loss, identity theft, or loss of trust, making safety relevant to every developer.

Standards and mindset

Industries rely on standards such as ISO 26262 (automotive), DO-178C (aerospace), and IEC 62304 (medical). Even when these don't apply, their mindset does: anticipate failure, design for resilience, and handle the unexpected gracefully.

Why systems fail ?

Many failures arise from situations developers never considered: "*that could never happen*".

Hardware faults, invalid inputs, and corner cases often lead to bugs.

Good design anticipates these events instead of assuming perfect conditions.

Undefined behaviors (UB) and reliability

Anthony connected software safety to undefined behavior (UB) in C++ - "behavior for which the standard imposes no requirements". In safety-critical code, UB is unacceptable. Preventing it means enforcing memory safety, input validation, and robust testing. Tools like static analysis, sanitizers, and hardened libraries help detect problems both at build time and runtime.

Contracts in C++26 as a safety leverage

C++26 introduces [contracts](#) for expressing preconditions and postconditions directly in code:

```
void func(auto* ptr)
    pre(ptr != nullptr)
{
    auto& ref = *ptr;
}
```

Contract violations represent real bugs. In safety-critical systems, handlers may trigger a state safe reset (depending on the use-case), while in others, they might choose to continue or abort cleanly.

Defense in depth, using appropriate strategy and tools

Anthony described safety as a [Swiss Cheese model](#): multiple imperfect layers together form a strong defense. No single measure is sufficient, but combined they create resilience thus trust.

- **Design checks:** prevent problems upstream and by-design.
 - Initialize all your variables
 - Use safe APIs: ranges `at` instead of `operator[]`, etc.
 - No raw pointers: prefer `std::span`, `std::string_view`, etc.
 - No indexing: prefer range-based for-loops
 - Use RAI to manage lifetime
 - Use contracts: [C++26 contracts](#), or even just documentation as it's better than nothing.
- Investigate bugs: debugger, stack-trace, core-dump, custom logs of states
- Hardened libraries
- **Testing:** problematic inputs like too long strings, negative quantity, SQL query injection attacks, etc.
- **Fuzzing:** [American Fuzzy Lop](#) ❤️
- **Static analysis tools:** [clang-tidy](#), [CodeQL](#), [SonarQube](#)
- Sanitizers

Meeting C++ 2025

Software and Safety - Anthony Williams



Defense in Depth

Use the "Swiss Cheese" approach:

- Design your system to minimize the potential for problems
- Use static analysis
- Test with potentially problematic input
- Fuzz test
- Use sanitizers and hardened libraries
- Use contracts



The Arene Base library

Woven by Toyota is developing [Arene](#) Base Library (*will be available in open-source soon*), a foundation for safety-critical software that meets [AUTOSAR](#) and [MISRA](#) guidelines (*so it's C++14/17 for now 🙄 tho*). It includes static containers (`inline_vector`, `inline_map`, `inline_string`, etc.), backports of modern feature like non-owning ranges (`span`, `string_view`, `mdspan`), and enforced precondition checks.

My takeaway

I really enjoyed this talk - it felt 100% legitimate to start the conference by addressing the current "C++ is *unsafe*" criticism/buzz head-on.

👉 Repeated, loud claims that "C++ is unsafe" contribute to a cognitive bias known as the **exposure bias** (related to the **familiarity principle**): when the same simplified message is heard often enough, it starts to be perceived as an unquestionable truth, regardless of context or nuance.

Over time, this shapes collective perception - especially among juniors who lack technical perspective, and among managers or decision-makers who may not have the expertise to challenge such claims. The result is not informed judgment, but decisions driven by repetition, fear, and oversimplification rather than by facts, trade-offs, and real-world constraints.

As this was my first time attending a talk from Anthony Williams, and I was genuinely impressed by how detailed yet fluid and approachable his presentation was.

He managed to cover a technically dense topic with clarity and purpose, setting a strong and confident tone for the rest of the conference.

Moreover, this topic strongly resonates with the engineering practices I advocate for on a daily basis in my professional work, which naturally reinforced the positions and recommendations I drew from these talks.

That said, experience has shown me how challenging it often is to raise awareness - especially among product owners and managers - about the importance of safety, testing, sound software design, code quality, modern

development practices, and **pragmatism** in general.

Too frequently, the focus shifts toward delivering something that **merely appears to work**, validated by a quick demo covering a narrow, happy-path use case, and pushed to production as fast as possible.

From my perspective, this does not constitute delivering value. On the contrary, it creates false assumptions about the product, fosters a bug-prone environment, and builds software on fragile foundations that will inevitably crumble over time. ☞ This concern is not a matter of developer preference or perfectionism: **our responsibility is to deliver durable value**, not unstable or misleadingly functional components. If only the simplest scenarios are "tested", and no appropriate tools are used, how can we confidently claim that the system actually works ?

💡 Addressing these issues later invariably costs far more than doing things properly from the start.

This is why, for me, **establishing a clear and shared Definition of Done (DoD)** - and actively educating all stakeholders around it - is essential for every project, product, and team. Peter Muldoon delivered an interesting talk about DoD last year, here at MeetingCpp: [Software Engineering Completeness : Knowing when you are Done and Why it matters ?](#).

💡 Too often, short-term delivery - followed by constant patching - are rewarded more visibly than long-term engineering quality. Shipping something quickly and repeatedly fixing issues in production can create the perception of efficiency: such developers are seen as fast, reactive, and indispensable when problems arise. This visibility aligns well with **managerial metrics that favor immediate outcomes and rapid response**.

In contrast, developers who invest time in thoughtful design, thorough testing, and safety-oriented practices tend to operate quietly. Their work prevents incidents rather than reacting to them, which makes its impact far less visible. As a result, they are sometimes perceived as slow or inefficient, despite delivering systems that are reliable: robust, maintainable, and resilient to change.

At its core, this is a question of effort visibility and incentives. **Reactive work is easy to observe; preventive work is easy to overlook**. Yet the real measure of value lies not in how quickly issues are patched, but in how rarely they occur - and in how sustainably the system can evolve over time.

I'll also make sure to check the [ETL library](#) and its [Arduino variant](#), as it offers a complementary solution tailored to best fit the embedded restrictions and needs.

Interesting/valuable inputs from the questions session:

- Why would one choose a normal/soften library over an hardened one ? ☞ Optimized for speed vs. optimized for safety.
- The vocabulary we use when talking to managers matters: **weaken/soften** libraries might sounds like something's wrong, while we'd rather underline what the tradeoffs are.

👤 **Sebastian Theophil**: To Err is Human: Robust Error Handling in C++26

📺 [Video](#), 📄 [slidedeck](#)

Everyday mistakes can make software brittle, unpredictable, or just plain frustrating.

Sebastian Theophil ([Think-cell](#)) delivered an excellent overview of the current landscape of error management in C++.

His message was clear: error handling is hard, often undervalued, and yet absolutely central to building trustworthy systems.

💡 **Key takeaway:** 5% of all error handlers handle 95% of problems.

The landscape of error handling: historical perspective

Sebastian began by contrasting reliability with error handling. Reliability demands considering faulty hardware, corrupted RAM, or broken sensors.

Error handling, in comparison, is "easy" - yet/still often done poorly.

Unexpected environments, unforeseen usage scenarios, and simple programmer mistakes are enough to cause cascades of bugs. Bad error handling accumulates technical debt and drains team efforts/time/motivation without delivering customer value.

- **C error handling:** errno, out parameters, ambiguous return codes
- **C++ exceptions:**
 - suffer from bad reputation: "glorified gotos", performances, banned from some codebases, unobvious control flow, etc.
 - yet have strengths: any line can fail and unwinding can clean up state automatically.
 - 💡 *Pro-tip: catch exceptions as locally as possible*

The modern stuffs: `std::expected`

Sebastian highlighted `std::expected` as the likely "next big thing" in C++ error handling:

- structured like return values
- **expressive** like exceptions
- and backed by a useful monadic API - like `std::optional` - (*transform, transform_error, and_then, or_else, etc.*).

This feature encourages **deliberate handling** without the unpredictability of exception-based control flow, which from my perspective best in many, many designs.

The modern stuffs: `contracts`

He then connected this with C++26 `contracts`, which offer customizable assertion mechanisms for expressing **preconditions** and **postconditions**.

This is a great replacement for "documented by not checked" runtime requirements, just like `constraints and concepts` did for `named requirements` at compile-time.

Bonus, `contracts` support exceptions (leading to contract violation), and are supported in `constexpr` contexts !

Contracts error handling strategy/semantic can be selected/customized at build-time, making them suitable for both debugging and hardened production builds.

- `ignore`: do nothing
- `observe`: call contract violation handler
- `enforce`: call contract violation handler and terminate

- `quick-enforce`: terminate

Crucially, they allow side effects such as logging - provided that these do not alter the program correctness.

```
int f(const int x)
    pre(x != 0)          // implicitly const
    post(r : r != x)    // r is const, x must be const
{
    contract_assert(0 <= x);
    return x + 1;
}

int f(const int x);    // redecl. without contract is ok
```

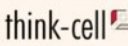
Meeting C++ 2025



CONTRACT CUSTOMIZATION

```

1 void handle_contract_violation(
2     contracts::contract_violation const& violation
3 ) {
4     if (violation.semantic() ==
5         contracts::evaluation_semantic::observe
6     ) {
7         CallMyErrorHandler(
8             violation.location().function_name(),
9             violation.location().line(),
10            violation.comment(),
11        );
12        return;
13    }
14    contracts::invoke_default_contract_violation_handler(
15        violation
16    );
17 }
```





Library hardening in C++26

The upcoming standards - see the [standard library hardening](#) proposal - bring enforced pre and postconditions to the standard library itself, turning many previously documentation-only requirements into runtime-validated checks.

Compiler vendors already provide early support, offering (partially) hardened library modes:

- GCC's `-fhardened`
- Clang's `LIBCPP_HARDENING_MODE`
- MSVC's `MSVC_STL_HARDENING`.

Practical rules for real-world code

Sebastian underlined that with so many options to do so, proper error-handling can be challenging.

- error handling must be easy, otherwise developers won't do it
- error handling must be tested
- error handling must be focused where it matters

He offered a set of rules shaped by **ThinkCell**'s own engineering experience:

- **Check everything:** every API call, every return value. Use consistent wrappers for various error-reporting mechanisms.
- Use `noexcept` by default and rely on `std::terminate` and custom terminate handlers for unexpected failures.
- **Assert aggressively**, even in release builds, and treat assertions as diagnostics - not reasons to exit.
- Design under the assumption that things usually work to keep the number of code paths manageable.
- When checks fail, **gather as much informations** as possible and **attempt to continue safely** (aligning with the "observe" contract mode).
- Focus efforts where the real problems are: historically, a small fraction of error handlers catch the vast majority of failures.

He also categorized error types:

- critical issues (null dereferences, unexpected API failures, assertion failures, etc.)
- untested scenarios
- "bad expected" (third-party bugs)
- and broken environments (weird configurations)

About **ThinkCell**'s error reporting

Sebastian concluded by describing ThinkCell's comprehensive out-of-process error reporting system.

Similar in spirit to [Google's Chromium Crashpad](#) but designed for all kinds of errors, it generates a context (gathering as many infos as possible) to then allow developers to best investigate issues, by suspending the failing process, generates OS-specific minidumps, uploads them (when/if user consent), performs server-side symbolication, callstacks localisation, and groups failures by emission sites.

This infrastructure lets pragmatically prioritize real-world issues and allocate effort efficiently, finding which areas of the codebase emits most problems (in perspective of time, versions, etc.).

My takeaway

I really appreciated how pragmatic and straightforward this talk was. I already use `std::expected` on daily basis, and it consistently proves its value - especially when wrapping legacy error-code APIs or when I need fine-grained, explicit reactions to likely error cases in a convenient and reliable way.

From my perspective, this style of error handling scales well in real products, and indeed is the "next big thing".

Contracts, on the other hand, are something I haven't tried in a professional setting yet, but I eagerly wait to. Hearing how they can be tuned from "debug aid" to "production hardening" made me genuinely craving to experiment more with them.

One of the strongest takeaways for me was how much real-world engineering depends on robust error handling: the customer-only bugs that never reproduce locally, the tiny set of handlers that end up catching the majority of problems, and the difference between merely showing an error message and actually fixing the underlying issue.

From my experience, far too many projects fall into the trap of only logging errors but never taking meaningful corrective action.

Logs pile up without being even grouped, analyzed, categorized, or qualified, and they become a poor substitute for real error handling - effectively forcing developers to painfully navigate through them by hand, which is inefficient, frustrating, and most of all, **not** error handling.

Related to this, I feel that many companies still struggle with non-reproducible or hard-to-reproduce issues (undefined behaviors, non-deterministic behavior, race conditions, unrecorded inputs, etc.), which becomes both expensive and exhausting to diagnose.

Using the right tools - debuggers, static analyzers, runtime sanitizers, and adequate/structured C++ error-handling mechanisms - is essential to improving team effectiveness and overall product reliability.

🔗 Sebastian's examples made it clear that investing in high-quality error handlers is a worthy strategy, over the entire lifetime of a product.

Additional resources: learn more about error handling

- [Peter Muldoon - Exceptionally Bad - The story on the misuse of exceptions and to do better](#)
- [Louis Dionne - ODR, libc++ hardening, Profiles and Contracts](#)
- [Security in C++ - Hardening Techniques From the Trenches - Louis Dionne - C++Now 2024](#)

👤 **Nicolai Josuttis**: Using `std::generator` in Practice

📺 Video, 📄 [slidedeck](#)

[Nicolai](#) gave an accessible tour of what `std::generator` brings to C++23, and how it finally sits on top of the coroutine machinery introduced in C++20.

I appreciated how he framed coroutines not as something exotic dark wizardry/witchcraft, but simply as stackless (in C++) - yet stateful -, suspendable/resumable functions.

I was surprised to learn that [Melvin Conway](#) coined the term as early as 1958.

It immediately reminded me of Peter Sommerlad's talk from last year here at the Meeting C++ conference - [Collective Amnesia?](#) - which explored how many supposedly "modern" ideas actually have deep roots in earlier programming models, along with several other invaluable insights for developers. (If you haven't seen it yet, go watch it !).

What coroutines are (and aren't)

C++ coroutines are **functions** that can **pause** and later **resume** exactly where they left off.

C++ coroutines are **stackless**: instead of relying on the stack, their **state** lives separately, which lets you write code that looks perfectly sequential while actually running asynchronously or producing values lazily-even infinite sequences if you want.

In practice, any function that includes any of the `co_yield`, `co_await`, or `co_return` keywords becomes a coroutine, it's that simple.

What `std::generator` brings

The talk really clicked when transitioning from the low-level coroutine model to the high-level `std::generator<T>` abstraction. Seeing generators used as range-like, lazy value producers made the

mechanics intuitive and ready-to-use:

- `begin()` starts running the coroutine until the first `co_yield`
- `operator*` accesses the yielded value
- `operator++` resumes execution until the next suspension point

He illustrated this with simple examples such as [Fibonacci sequence](#) generation and building a `concurrent_queue` reader coroutine.

The contrast with old user-defined stateful functors made it obvious how much cleaner and straight-forward generators are.

```
std::generator<std::string> coro() {
    std::array coll{ "tic", "tac", "toe" };
    for (int i = 0; i < coll.size(); ++i)
        co_yield coll[i]; // suspend with value
}

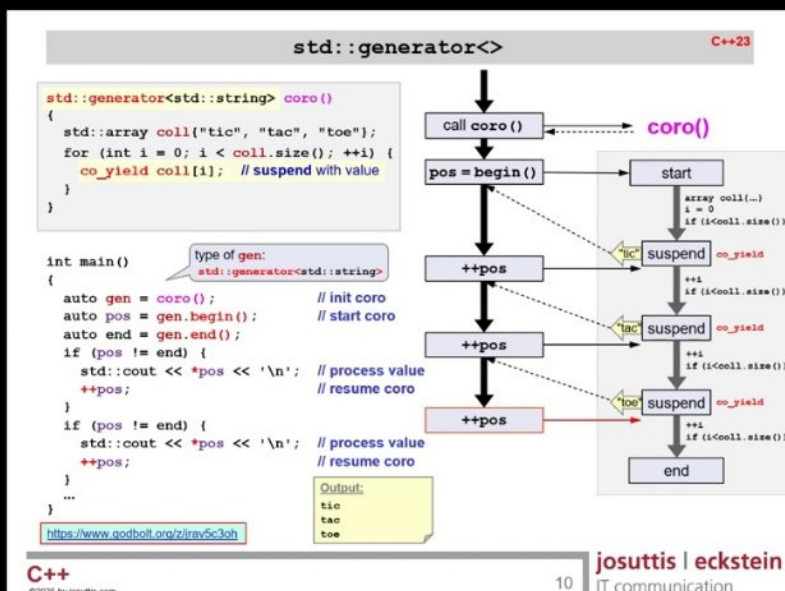
for (auto value: coro()){
    std::println(value);
}
```

Output:

```
tic
tac
toe
```

See example [here on godbolt](#)

Meeting C++ 2025



```

#include <cstdint>
#include <generator>
#include <print>

std::generator<std::uint64_t> fibonacci() {
    std::uint64_t a = 0, b = 1;
    while (true) {
        co_yield a;
        auto next = a + b;
        a = b;
        b = next;
    }
}

auto view = fibonacci() | std::views::enumerate | std::views::take(20);
for (auto element : view) {
    const auto & [index, value] = element;
    std::println("{:2} -> {}", index, value);
}

```

Meeting C++ 2025



std::generator<> versus Class C++23

```

std::generator<long> fibonacci(int num)
{
    long a = 1;
    long b = 1;
    co_yield a; // first value
    co_yield b; // second value
    for (int i = 0; i < num-2; ++i) {
        auto sum = a + b;
        co_yield sum; // third to last value
        a = std::exchange(b, sum);
    }
}

for (auto val : fibonacci(20)) {
    std::cout << val << '\n';
}

```

<https://www.godbolt.org/z/veM9eGa3g>

```

class Fibonacci {
private:
    int num;
    long a = 1, b = 1;
    int i;
public:
    Fibonacci(int n) : num(n), i(0) {}
    std::optional<long> nextValue() {
        ++i;
        if (i <= 2) {
            return 1; // first and second value
        }
        if (i <= num) {
            long sum = a + b;
            a = std::exchange(b, sum);
            return sum; // third to last value
        }
        return std::nullopt; // no more value
    }
};

Fibonacci fibo(20);
while (auto optVal = fibo.nextValue()) {
    std::cout << *optVal << '\n';
}

```

C++ ©2023 by josuttis.com

13 josuttis | eckstein IT communication



Pitfalls and caveats

Nicolai didn't shy away from discussing the limitations, and I genuinely appreciated that. Few things are more frustrating than seeing a conference talk showcase something impressive, only to have it fall apart the moment you try to use it in the real world, immediately falling from the "peak of mount Stupid" down to the "valley of despair" ([Dunning-Kruger effect](#)).

Important points:

- Generators are **single-pass**: iterating more than once is **undefined behaviors** (like stateful views).
- **Lifetime** matter: you shouldn't take parameters by reference if they might not outlive suspension.
- Yielded values often behave like references.
- You cannot iterate a generator when the object is const, since `begin()` mutates state.
- `std::generator<>` is non-copyable.
- Error management: exceptions propagate normally through the generator, back to the caller.

So one should not assume container semantics.

My takeaway

I especially enjoyed this talk from a meta perspective: the way the speaker presented and demonstrated the feature made it feel accessible, simple and approachable, without romanticizing it - just a clear, pragmatic view.

Nicolai's explanations further reinforced why this abstraction can be so useful/powerful. From a design standpoint, it cleanly decouples value generation from consumption in a way that feels natural in contemporary C++.

The boilerplate required for custom coroutine types is still heavy - my own experience is around 100 lines for something usable - so I'm genuinely looking forward to future coroutine helpers in the standard.

I'm also especially curious about how well this whole model integrates with concurrency and networking scenarios. The idea of coroutine-based pipelines for network message handling or sensor data acquisition is very compelling.

Despite coroutines having been introduced in C++20, I definitely haven't experimented enough with them yet. This talk left me wanting to spend a lot more time exploring what they can do.

Additional resources: learn more about coroutines

-  [C++ Coroutines Demystified - Phil Nash - ACCU 2025](#)
- [C++ Coroutines from scratch - Phil Nash - Meeting C++ 2022](#)
- [C++20's Coroutines for Beginners - Andreas Fertig - CppCon 2022](#)
- And this very fun, simple, and short video by [Sy Brand: Coroutines Explained with My Cat](#)

Sandor Dargo: Code Reviews: Building Better Code and Stronger Teams

 [Video](#),  [slidedeck](#)

Sandor's talk focused on a topic that every developer deals with **daily**, yet few teams truly master: **code reviews**.

What I appreciated is that he didn't treat code reviews as a mere technical gate, but as a human process - one that can teach, uplift, frustrate, or damage depending on how it's handled. In that way, it reminds me of [Kate Gregory's talk at CPPP 2019 Emotional Code](#), which is IMO a masterpiece.

He explored why reviews matter, why they can go wrong, and how to make them constructive both for the codebase and the people involved.

The value of code reviews

Sandor broke down code reviews into three complementary roles:

- **Quality assurance:** catching inconsistencies, design flaws, unclear logic, architectural drift, or naming issues.
Not a silver bullet, but an important final human check after all the automation.
- **Knowledge sharing:** spreading API familiarity, explaining context, avoiding the "only the author knows this" trap.
- **Mentoring opportunity:** especially for juniors, reviews can clarify the "why", not just the "what", and help them grow with empathy.

He also highlighted review formats: **synchronous** (pair or mob programming, dedicated review meetings) vs. **asynchronous** (pull requests).

Pair/mob programming offer immediate feedback and shared understanding, while PRs encourage reflection and diverse viewpoints.

Common arguments and misconceptions

- ☞ **"Reviews slow us down":** true in the short term, but they prevent costly rework and architectural decay.
- **"Reviews don't catch bugs":** also true. They're better at catching design mistakes, complexity, readability issues, or misunderstandings - the subtle stuff tests rarely detect.
- Tradeoffs exist, but the long-term payoff is healthier code and stronger teams.

The human aspects

Sandor underlined how important **empathy** and **communication** are:

- Tone is easily misinterpreted in writing.
- Comments should avoid blame, judgement, or condescending phrasing.
- Excessive commenting or unclear prioritization overwhelms authors.
- Poorly prepared PRs (too large, red, unrelated changes, no description) sabotage the process.

His summary of common pitfalls really resonated: late feedback, nit-picking, bossy language, unclear blocking status/reason, and comments lacking explanation.

Sandor's AIR methodology

Sandor wrapped up with his own structure for writing comments:

- **A - Action:** what the individual contributor (IC) should do.
- **I - Information:** why it matters.
- **R - Reference:** where to learn more (link to guideline, documentation, etc.) that best justify the feedback

Which results in a simple, predictable shape that keeps feedback actionable, polite, and grounded.

Interestingly, This reminds me of the [Gherkin language](#) in [Cucumber](#), used for **behavioral specifications** (scenario, Given, When, Then),

that I use on daily basis to create backlog entries (features specs, issues, bugs, etc.).

The key idea here is to produce a predictable, narrative format that helps reduce ambiguity and clarifies intent and legitimacy.

Meeting C++ 2025



Code Reviews: building better code and stronger teams - Sandor Dargo

Quick Recap: The AIR Formula

Component	Purpose	Example
Action	What to do	"Please consider renaming"
Information	Why it matters	"To clarify its meaning"
Reference	Learn more or justify	"See our team style guide..."



My takeaway

I found the idea of delivering a talk about this topic very relevant. For the past decade, a large portion of my role as a lead-developer has revolved around reviewing code, coaching colleagues, and helping teams to deliver value.

So hearing Sandor frame reviews as both a technical practice and a human one strongly resonated with me.

That said, I wished he had gone a bit deeper into the "real-world" part of timely reviews.

He mentioned that PRs should be reviewed shortly after they're created and should not linger for days, but the talk didn't offer much practical advice for avoiding the ping-pong effect - especially common with junior developers or contributors unfamiliar with guidelines (or reluctant to follow them).

In my experience, even when the team has clear rules, some PRs naturally drag and slow the entire delivery pipeline.

Many stakeholders and managers still view code reviews as something that slows down delivery. As a lead developer, part of my job is to help them understand why this work is essential - but it's not always easy. Just like testing, error handling, avoiding undefined behaviors, and building resilient designs, the value of thorough reviews often only becomes obvious when things go wrong.

And who hasn't heard an IC complain to a manager: *"My PR works ! Why is the lead dev blocking it ? I just want to deliver value and reach my monthly performance goal"* . But **protecting a codebase** from accumulating exponential, untracked technical debt is hard to measure, yet absolutely vital. Solid reviews prevent the system from sliding toward a "point of no return," where changes become prohibitively expensive - *or nearly impossible* - and development costs rise dramatically.

I also take a different stance on PR presentations. Unlike Sandor, I avoid *"let me walk you through the code"* meetings, because they tend to introduce **biases**, anchoring reviewers to the author's narrative, and reducing the chance of catching misunderstandings.

Instead, I prefer/favor clear PR summaries and design diagrams.

Reviewers's role then is to evaluate the possible delta between:

- what was requested
- what the author thinks they delivered
- and what is actually in the PR's diff

When that delta is too large, repetitive, or highlights a recurring technical error, then I second my review with a coaching sessions in order to avoid endless round-trips - thus speed-up the overall value delivery process.

Bonus: How I personally do code reviews

Over the years I refined my own approach:

- **Use the right tools:** for instance if you're using Microsoft VSCode then you really might want to give the [microsoft/vscode-pull-request-github](#) extension a try, so code navigation is available to best understand the code context (types, values, call hierarchy, namespaces, etc.).
- Structure comments consistently, for clarity and expectations management:

 |
  |
  |
 ? |
  [categories...] Description
 Reason/context, with references
 Action requested

◦ traffic lights

-  Not blocking
-  Consider fixing. Add a tech-debt entry to the backlog otherwise.
-  Blocking
- ? Blocking: question (I did not understand something)
-  Blocking: obviously a bug

◦ categories

- bug
 - business (likely to degrade the users experience)
 - readability
 - naming
 - cognitive-complexity
 - design
 - performances
 - etc.
- Add reference links: the C++ standard relevant section, internal documentation, guidelines, etc.
 - If you're unsure about something, that uncertainty itself is - *and will remain* - a maintainability issue, as the IC intent is not clearly translated into code. Never hesitate to ask for clarification.
 - Be kind, positive and encouraging. Acknowledge the efforts.
 - Use some emojis like 😊 😄 🤔 so the user knows you're not mad.
 - Include a hint of positive reinforcement in your code-review summaries (👏 🎉), especially when an IC successfully applied techniques covered in recent training/mentoring sessions.

This structure keeps expectations clear and helps to keep reviews consistent and fair across all contributors.

Conclusion

Overall, Sandor's talk was a strong reminder that code reviews are not just about code quality - they're a social contract within the team.

When done well, they teach, support, and build trust.

When done poorly, they erode relationships.

I left the talk with a renewed motivation to continue investing in review culture and helping teams make the most of it.

Anders Schau Knatten The Two Memory Models

 [Video](#),  [slidedeck](#)

What are the two memory models ?

The programming language memory model

- What programming languages (C++, C, Rust, etc.) guarantee
- Abstract over the physical reality
- Data race rules
- What are "observable behaviors"
- How atomics provide/guarantee synchronization and ordering
- Which reorderings the compilers are allowed to perform

The CPU/architecture memory model

- Which load/store orders hardware may produce
- How pipelines, store buffers, caches, and micro-ops reorder things
- How architectures differ (x86 stricter, ARM/RISC-V more relaxed)

That duality - language vs. hardware - was the core concept of the talk, and understanding how they may not match each other intuitively is key to writing correct concurrent code and avoiding disasters.

At language-level

The standard describes an abstract machine, not real execution.

- Compilers may reorder freely as long as observable behavior stays the same.
- Data races are UBs
- `seq_cst` gives one global total order
- Acquire/release synchronizes like a lightweight mutex handoff
- Relaxed atomics don't give cross-thread ordering.

At CPU-level

CPUs reorder far more than we imagine.

- x86 never reorders stores, while ARM and RISC-V can reorder stores to different addresses
- Store buffers, pipelines, coalescing, etc. all affect observable ordering
-  Looking at assembly doesn't prove your concurrent code is correct

Mental model for developers

Not "did instruction *X* run before *Y*?", but "does *X* happen-before *Y*?"

Meeting C++ 2025

SQUAREHEAD

Woven by TOYOTA

TSO EXAMPLE

Thread 1	Memory order	Thread 2
store x 1	load y	store y 1
load y	store y 1	load x
	load x	
	store x 1	

My takeaway

👉 Disclaimer & overall impression

This talk absolutely melted my brain - in the best possible way. While I've always found memory ordering intriguing, this session made it clear that I've only been barely scratching the surface, which is so exciting. I'll definitely **need at least one rewatch** to get a slice of such a dense yet important topic: Anders covered both the language-level memory model and the CPU memory model, how they interact, and why they're fundamentally different mental machines.

Even though I already use `std::atomic` regularly in simple use-cases, this talk left me with a much stronger desire to dig deeper into non-trivial patterns, lock-free reasoning, and the real consequences of memory ordering on correctness and performance.

In a nutshell, this made me realize how shallow my current mental model is around:

- Subtle memory orderings
- Cross-architecture behavior resilience/consistency
- How atomics map to actual hardware
- Why I should dig deeper into lock-free algorithms/patterns

I love the feeling of being humbled - *if not a bit dumb* - by a talk - it means there's a whole new frontier to explore, thus an exciting room to level up! 🤖

Additional resources

- [Anders's blog](#)
- [CppQuizz](#)



11/07/2025

👤 **Jens Weller** - Good Morning Meeting C++

Jens opened day 2 sharing how he designed and implemented - using [Qt](#) - this year's conference merchs design for [T-shirts](#), [hoodies](#) ([Spreadshirt](#)) and [calendars](#) ([Etsy](#)).



He also talked about his ongoing projects, including the Meeting C++ CMS and a small game-like ecosystem simulation with plants and animals acting as agents - which resonated with me, since creating entities simulations, implementing their behaviors and watching them evolve is a genuine passion of mine. From my perspective, it's a great way to learn (about ECS, determinism, reproducibility vs. repeatability, software-design, performances, rendering, etc.) while having lots of fun.

From there, he touched on design perspectives like SOA vs. AOS/OOP, and mentioned that the schedule had been slightly adjusted due to Victor Ciura canceling his talk. Jens also highlighted the cool stands this year: Qt's booth with automotive automation demos, and Flow Traders's.

He wrapped up by previewing next week's post-conference workshops:

- C++ software design - Klaus Iglberger
- (canceled) multi-threading and concurrency in C++: a workshop full of live coding
- AVX vectorization workshop - Ivica Bogosavljevic
- Intermediate to advanced C++ - Slobodan Dmitrovic
- MISRA C++ 2023 guidelines workshop - Richard Kaiser

My takeaway

I'll definitely watch the trendy talk "[More Speed & Simplicity: Practical Data-Oriented Design in C++](#)" - [Vittorio Romeo - CppCon 2025 Keynote](#) to dig deeper into the SOA vs. AOS topic, and DOD in general.

And like always, hearing about an ecosystem simulation project reignited my own itch to spin up a fresh entity-based experiment 🤖. *So many things to do, so little time though.*

👤 **Frances Buontempo** - Center Keynote - Stoopid Questions

📺 [Video](#), 📄 [sliddeck](#)

France's keynote was a deep, reflective dive into learning, teaching, curiosity, and how important the role of so-called "stupid" questions is.

She opened by reminding us that **everyone learns differently**, and that - despite the internet and LLMs - real learning still takes time, efforts, and explorations.

🔗 We're rarely taught **how to learn** or **how to teach**, yet both shape our careers, and ultimately lives.

A recurring theme was the importance of **asking questions**: during talks, at work, in classrooms, even when something seems obvious.

This is what I call **proactive listening**: while watching videos, I regularly find myself talking to the screen, anticipating the points that the speaker may/will address next.

But asking questions directly requires **psychological safety** and confidence - challenging a speaker, a book, a teacher, or a colleague can feel intimidating, even though they can be wrong too.

In any case, it's always worth it: if you have a question, then it means that you have not understood some points as the speaker intended – and probably other participants as well.

France highlighted blockers that slow learning: school baggage, neurodiversity challenges, bad teachers, or simply feeling stuck.

She offered practical solutions/tools like rubber-ducking, shrinking the problem, **exploring "what-ifs"**, switching to a related task, or simply asking for help.

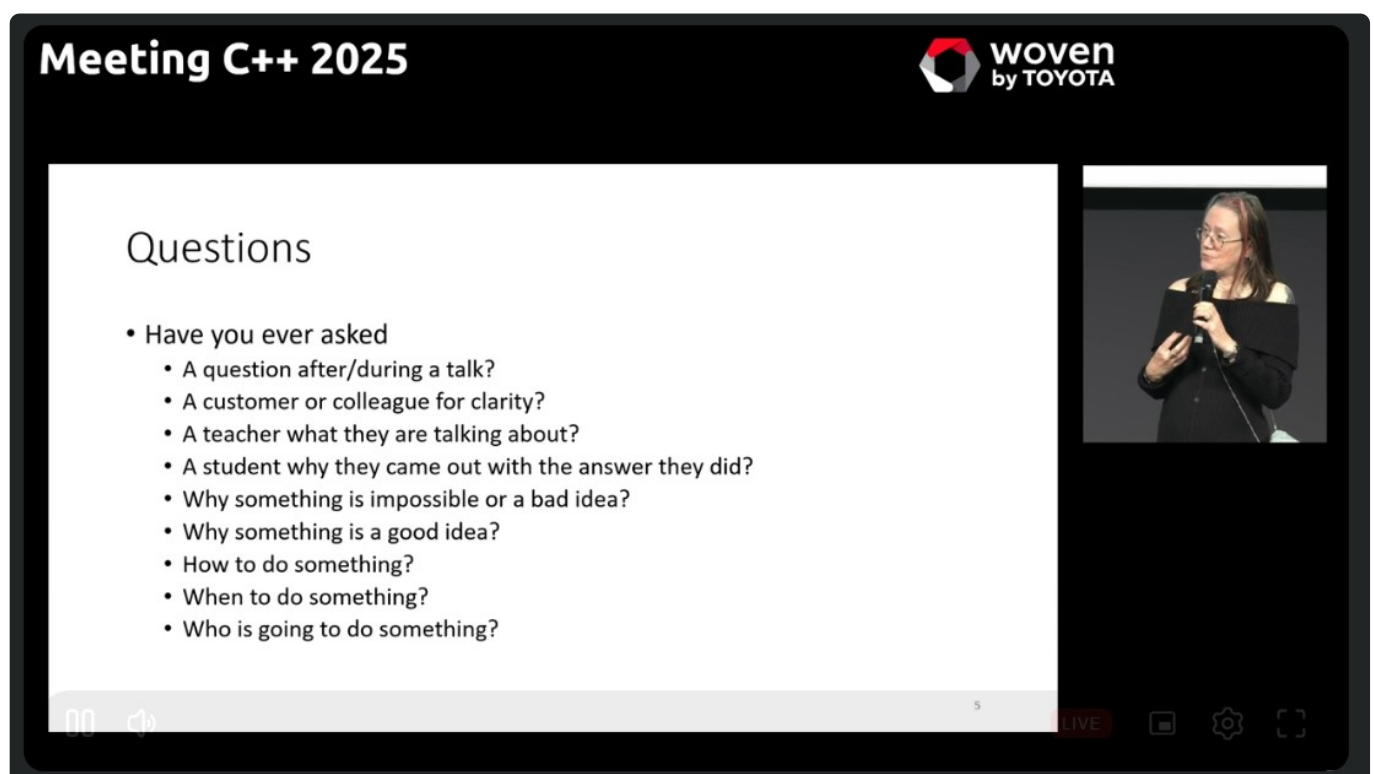
And just as important: **learning must be a fun process** - through games, surprises, experimentation, and small wins.

She also touched on emotions in teaching and learning - frustration, boredom, non-linear progress - and the need for teachers to stay patient and adaptive. Finally, she contrasted human learning with the limitations of machines: LLMs don't learn, they don't understand, and they often hallucinate.

This should often prompt us to ask more relevant questions, with the aim of finding a potentially relevant answer. Which, at best, is a waste of time, and at worst, a plain loss.

France closed with challenges for everyone:

- Ask a question !
- Try something you feel terrible at
- Read a book
- Give a talk.



Meeting C++ 2025 **woven by TOYOTA**

Questions

- Have you ever asked
 - A question after/during a talk?
 - A customer or colleague for clarity?
 - A teacher what they are talking about?
 - A student why they came out with the answer they did?
 - Why something is impossible or a bad idea?
 - Why something is a good idea?
 - How to do something?
 - When to do something?
 - Who is going to do something?

5

LIVE

My takeaway

For me, learning is effective over the long term only when it remains **motivating and meaningful**, and when it is treated as a daily practice. It should not be perceived as a constant effort or burden, but rather as dedicated time that one genuinely enjoys and values.

In practice, I have encountered many developers who become blocked not for technical reasons, but due to diminished confidence or reluctance to ask for help. Regardless of whether they identify as "junior" or not, some remain stuck for extended periods, choosing isolation over collaboration.

In a fast-evolving ecosystem such as C++, this mindset can significantly hinder both individual growth and overall team effectiveness.

I have also observed a recurring pattern - in schools, universities, and many companies - where those who ask questions are often perceived as lacking competence, particularly on technical topics. Even requesting clarifications or additional specification details may be interpreted as a lack of autonomy. Such perceptions directly undermine psychological safety and discourage open communication.

 Ultimately, a learning culture that equates asking questions with incompetence is **counterproductive and toxic**.

 Sustainable expertise emerges not from silent struggle, misplaced pride, or biased ICs KPIs, but from **curiosity, dialogue, and the confidence to seek understanding**.

From my experience, **side-projects** work great and are often under-rated. I always encourage my ICs - *and juniors in particular* - to build something fun: games, simulations, little experiments, etc. because **real problems expose your limits**: poor design, performance bottlenecks, spaghetti code, poorly evaluated trade-offs, quick-win-but-slow-loose, etc.

Hitting those walls creates an authentic need to understand and improve, and that kind of learning sticks far better than passive theory, endless katas, or rote exercises.

While this requires time and courage, as confronting failures avoids biases and luring yourself into a false sense of understanding.

My own routine reflects this: I spend at least an hour every day - more on weekends - coding on side-projects, reading standard/books/blogs, browsing GitHub, and/or watching talks.

It's the balance of practice and passive input that works best for me. Ultimately, as Frances emphasized, everyone needs to discover how they learn before anything else.

 **Psychological safety** is essential: teams need a kind, supportive environment where people can rely on each others without fear of judgment. Misplaced ego or pride only slows everyone down and can even lead to disasters.

 I also deeply regret that many companies still treat developer trainings - especially for juniors - as a waste of time.

This fuels the myth of the so-called "*full-stack dev-sec-ops 10x engineer*" who is magically productive straight out of school, and can perform/achieve any task autonomously/independently in a short amount of time. Software development is a team sport that requires collaboration, shared understanding, and **deliberate investment in people**.

 Training is not a cost to be minimized, but a prerequisite for building resilient teams and sustainable software.

🔗 Real growth requires time, guidance, and a learning culture.

In that spirit, I asked France about something I often struggle with: how to evaluate retention when designing a course, as each step depends on the previous one.

Her answer was simple and practical:

- If some point takes longer to teach than expected, it's a sign something's wrong.
- Make it explicit that everyone must understand before moving on
- And (of course) constantly encourage questions

This strongly resonates with my own experience: meaningful practice, supportive peers, and an environment where questions are welcomed are far more valuable than any amount of passive learning.

👤 Jonathan Müller - Cache-Friendly C++

📺 Video, 📄 slidedeck

Jonathan Müller - also known as *foonathan* - gave an accessible overview about what designing cache-friendly C++ code really means, and why `std::vector` in particular should almost always be your default container. This talk is essentially a modern, refined follow-up to his 2018 "Writing Cache-Friendly C++" presentation here back at MeetingCpp.

He started from first principles: CPUs are extremely fast, but memory is not.

Accessing main memory way slower than accessing data already stored in a CPU cache.

This is why CPUs rely on several layers of caches (L1, L2, L3): L1 is tiny but extremely fast, L2 is slower but larger, and L3 is larger still but slower again - yet still much faster than accessing memory.

To run fast code, your data needs to reach these caches efficiently. Which starts with designing cache-friendly code **and** benchmarking such designs for your targeted architecture.

Cache-friendly code: why does it matter ?

Cache-friendly programming is fundamentally about data locality:

- Keeping related data contiguous and in a predictable layout
- Minimizing memory footprint so more data fits inside a single cache line
- Accessing memory sequentially whenever possible.

If data is well packed and accessed in order, CPUs can prefetch it efficiently and process it with minimal [cache misses](#).

Data structures matter

This is why a sorted [std::vector](#) often beats [std::set](#) or [std::unordered_set](#):

- `std::vector` is contiguous and thus predictable
- Tree-based containers force pointer chasing and scattered memory access
- Open-addressing hash tables do better than node-based ones for the same reason.

Type size and layout affect cache use

Jonathan highlighted how even small layout decisions influence cache footprint:

- Reordering struct members reduces padding
- Misaligned or sparsely laid-out data wastes precious cache space
- Bitfields can sometimes hurt due to extra work the compiler must generate

Optimize type size: Re-ordering members

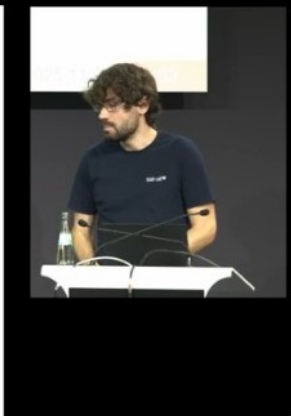
| think-cell

```
struct Message { uint8_t; uint32_t; unsigned char*; uint16_t; };
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
u8				uint32_t				unsigned char*								uint16_t							

```
struct Message { unsigned char*; uint32_t; uint16_t; uint8_t; };
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
unsigned char*								uint32_t				uint16_t		u8									



Prefetching & cache lines

- CPUs move memory in cache lines, not bytes.
- Sequential access allows the hardware to prefetch aggressively.
- Random access breaks that pipeline and forces costly fetches from DRAM.

Code is data too

CPU instructions are stored in memory

- Avoid long branches
- Avoid indirect jumps
- Minimize the size of hot code

Jonathan mentioned that sum then increments rather than increments the sum can be significantly - if not, way - faster on some CPU, which feels kinda bizard.

Takeways

| think-cell

Cache-friendly data access patterns

- Access data in linear order
- Avoid pointer chasing
- Minimize the size of data

Cache-friendly code patterns:

- Avoid long branches
- Avoid indirect jumps
- Minimize the size of hot code



Data-Oriented Design (DOD)

He emphasized thinking in terms of algorithms and data flows, not objects.

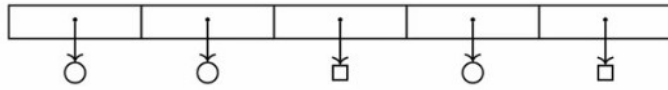
- Process N items together
- Re-organize your data to match access patterns
- Leverage SoA - rather than AoS - layouts when appropriate

Representation of heterogeneous data



Boxed

```
std::vector<std::unique_ptr<Shape>>
```



Tagged union

```
std::vector<std::variant<Circle, Square>>
```

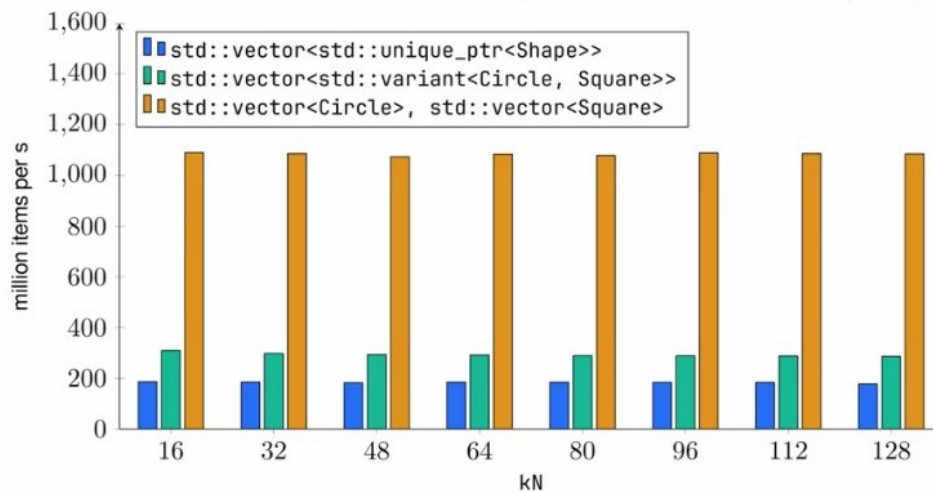


Multiple homogeneous arrays

```
std::vector<Circle>, std::vector<Square>
```



Multiple homogeneous arrays are significantly faster



Multicore pitfalls

False sharing - two threads modifying unrelated values that live in the same cache line - can devastate scalability. Padding structs with `std::hardware_destructive_interference_size` - *minimum offset between two objects to avoid false sharing* - helps avoid this.

My takeaway

This talk reinforced how critical it is - especially in performance-sensitive code - to think consciously about how data lives in memory. Choosing containers, controlling alignment, avoiding unnecessary pointer dereferencing: these things matter more than most teams realize. Redesigning an entire system because of performance issues is often a long and costly process, so it's far better to forecast such problems and address them early.

I've long favored cache-friendly containers like `std::array` and `std::vector`, and this talk validated that instinct.

`std::array` remains one of my favorite containers because it behaves both like a `tuple` and a `range` while keeping data contiguous 🤖. From experience, `std::map` and `std::set` often loose to a simple `std::vector<std::pair<K, M>>` because contiguous memory access beats pointer-heavy tree traversal in many real workloads.

Also, I remember struggling with performances issues when digging into [trie containers](#) implementations, which is from my perspective a meaningful illustration of the "size in memory vs. speed" tradeoff.

Jonathan's insights also made me want to explore [Data-Oriented Design](#) more deeply for large-scale or simulation-style workloads - where data locality matters a lot.

I'll definitely continue digging into cache-friendly programming. It still feels underrated in the industry: too many teams default to pointer-rich OOP AoS designs ("everything is a `std::shared_ptr`") without considering the cost on locality, cache lines, and ultimately performances.

This talk was a great reminder to structure programs around access patterns and data, not just obvious-for-humans abstractions.

Andreas Fertig - Embedded-Friendly C++: Features That Make a Difference

 [Video](#),  [slidedeck](#)

Andreas links: coppinsights.io, [Youtube channel](#), [talks slide-decks](#).

The talk was framed around a simple, relatable scenario: building embedded lightbulb controllers that communicate over a network. Using this example, Andreas walked through a series of common embedded tasks - parsing input, handling raw bytes, dealing with endianness, interfacing with C APIs, and managing memory safely - and showed how contemporain C++ (C++20/23/26) provides clean, safe, and expressive solutions.

Content

Andreas demonstrated a broad range of features, each tied to a concrete use-case:

- **Safe parsing & input handling:** [std::span](#), [std::expected](#), [std::from_chars](#), and [std::to_array](#).
- **Expressive syntax:** [user-defined string literals](#) to improve readability.
- **Safe byte manipulation:** [std::bit_cast](#) to reinterpret data without undefined behavior (rather than plain [reinterpret_cast](#)).
- **Enum handling:** [std::to_underlying](#), [std::underlying_type](#).
- **Dealing with network data:** [std::start_lifetime_as](#) to reinterpret packet bytes safely (as opposed to the common [reinterpret_cast](#) approach).
- **Endianness:** [std::endian](#) and [std::byteswap](#).
- **Resource management:** [std::unique_ptr](#) with custom deleters for wrapping legacy APIs.
- **Modern compile-time features:** [constexpr](#), [constexpr](#), [constexpr](#), [if constexpr](#), and static [constexpr](#) local variables.

Overall, the talk was a tour of "modern tools solving common/old embedded problems" shown through minimal, digestible and realistic/plausible examples.

Converting a float to its byte representation using `std::bit_cast`

```

1 constexpr FP16 toFP16(float f)
2 {
3     const uint32_t bits{std::bit_cast<uint32_t>(f)};
4     const uint32_t sign{(bits >> 16) & 0x8000u};
5     const uint32_t exp{((bits >> 23) & 0xFFu) - 127u + 15u};
6     const uint32_t mant{(bits & 0x7FFFFFu) >> 13u};
7
8     // Check for underflow
9     if(exp <= 0) { return {static_cast<uint16_t>(sign)}; }
10
11    // Check for overflow
12    if(exp >= 0x1Fu) { return {static_cast<uint16_t>(sign | 0x7C00u)}; }
13
14    return {static_cast<uint16_t>(sign | (exp << 10u) | mant)};
15 }

```

C++20

Receiving data from the network

```

1 void Receive(std::span<const unsigned char> data)
2 {
3     // Use bounds checked at() in C++26
4     if(Protocol::Update == Protocol{data[0]}) {
5         :-
6         Process(*std::start_lifetime_as<LightBulbControl>(data.subspan(2, sizeof(LightBulbControl)).data()));
7     }
8 }

```

C++23

My takeaway

As always with Andreas, I was struck by how incredibly didactic and clear he is 🤖.

His teaching style is calm, peaceful, and remarkably effective - a masterclass for any C++ trainer like me.

I especially appreciated how every feature was grounded in a real embedded need, showing not just what modern C++ offers, but why it matters in constrained environments.

Most - if not all - of the examples reminded me of code I've seen many times in peer-reviews. In my experience, especially when related to the embedded world, contributors still tend to write old-style, error-prone code using patterns like `reinterpret_cast`, manual bit manipulations, index-based buffer manipulations, unions, and so on.

This coding style, in my view, often originates from legacy teaching approaches that are still widespread today. These approaches emphasize rigid rules, prescriptive "do and don't" lists, katas, and patterns. When developers are taught only how to use a hammer, every problem inevitably starts to look like a nail. This results in two major issues: anchoring knowledge at some point in time rather than keep learning modern incoming features/techniques, and inhibiting creativity, flexibility, and ultimately pragmatism. This reduces software development to a simplistic pattern-matching exercise, where developers merely choose between predefined solutions instead of engaging in analysis, design, and trade-off evaluation.

The result is code - thus, PR - that is long, hard to read, difficult to maintain, and ultimately less safe and less reliable.

A few personal reflections:

- Like always: **contemporain C++ really simplifies** codebases, including the ones related to embedded. Tasks that once required brittle tricks-casting bytes, parsing raw packets, converting enums-now have safe, standardized solutions.

- The combination of `std::span` + `std::expected` + `std::from_chars` is something I already promote at work. It's such a clean way to handle raw data and error-cases.
- Compile-time features (`constexpr`, `constinit`, `if-constexpr`) offer a great way to enforce correctness without runtime cost.
 - I particularly appreciated the `if not constexpr { std::println("", ...); }` trick that I'll make sure to reuse, so I'll extend my `constexpr` tests to functions that might emit logs and user-notifications.
- `std::unique_ptr` with **custom deleters** remains an easy and elegant bridge to legacy C APIs.

I'll make sure to dig into the presented features which I'm not familiar with yet, like `std::start_lifetime_as` and `std::has_unique_object_representations`.

I came away with several insights I can apply right away, and an even greater appreciation for Andreas's teaching style. His approach genuinely inspired me to revisit my own training slide-decks, so I can make my courses more grounded in real-world needs and even more accessible; which might increase my **trainings retention rate**.

Daniela Engert - Towards Safety and Security in C++26

 Video,  [slidedeck](#)

Disclaimer: I chose to attend Daniela Engert's talk instead of Evgenii Seliverstov's "Sanitize for your Sanity: Sanitizers tools for Modern C++", but I'll definitely watch Evgenii's session as soon as the recording becomes available.

"Simplicity is a prerequisite for reliability", [Edsger W. Dijkstra](#).

Content

Daniela's session fit perfectly within this year's recurring theme: **tackling fears, misconceptions, and outdated beliefs** about C++ being inherently unsafe. She grounded the discussion in real data: around 70% of [Microsoft's](#) and [Chromium's](#) CVEs come from memory safety issues, and the top CWE categories are exactly the usual out-of-bounds read/write access, use-after-free, null dereferences, integer overflows.

She reframed safety in its broad sense: memory safety, functional safety, system safety. And she walked through how contemporary C++ has been steadily strengthening its safety model.

Key points included:

- **Undefined behaviors** are everywhere (about 90 occurrences in language-specific part of the standard), but **UBs exist for a reason**: so compilers can optimize to generate faster instructions, not to punish developers.
- **Constant evaluation** is becoming a cornerstone of safety: all UBs are detected and refused during constant evaluation.
- **C++ is getting safer with every standard**, every 3 years. `constexpr` support keeps expanding (remember [Hana Dusikova](#) proposal last winter)
- **Standard library hardening (P3471)** brings runtime-checked preconditions to the STL when enabled.
- **Trade-off: safety vs. performance** - but if performance allows it, hardening is an immediate win.
- **Contract support & partial program correctness (P2900)** opens the door to precise diagnostics and better program correctness, coming with

- `pre`, `post`, and `contract_assert`,
- and policy/semantics like `ignore`, `observe`, `enforce`, and `quick_enforce`.
- **Observable behavior & checkpoints (P1494)** prevent compilers from reordering checks in ways that could undermine contract guarantees.
- **Beyond C++26**: profiles like [P3081 \(Core safety profiles for C++26\)](#) propose opt-in enforcement of type, bounds, lifetime and arithmetic, potentially moving C++ toward even safer subsets of the language.

Overall, her talk showed a clear trajectory: the language and the standard library are evolving toward practical, opt-in safety - without abandoning performance.

```

1 import std;
2
3 int main(int argc, char*[]) {
4     std::println("hardened STL: {}", static_cast<bool>(_MSVC_STL_HARDENING));
5
6     std::vector v(std::max(argc, 1), argc);
7     std::println("v[3] = {}", v[3]);
8     std::println("v[-1] = {}", v[-1]);
9 }

```

MS-STL version 202502

just as ever

```

1 > cl test.cpp
2 > test.exe
3 hardened STL: false
4 v[3] = 9673
5 v[-1] = -1811929271
6
7 // program ended with return value 0

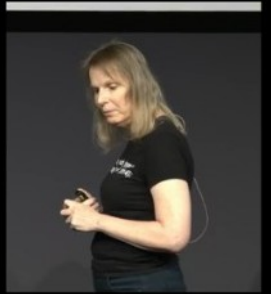
```

with hardening

```

1 > cl test.cpp
2 > test.exe
3 hardened STL: true
4
5 // program execution stopped here
6 // with error code 0xC0000409
7 // i.e. STATUS_FAIL_FAST_EXCEPTION

```



```

1 // somewhere in the program source
2
3 int func(int x)           // function parameters are *immutable* in pre/post assertions!
4     pre (x <= 1)           // a precondition assertion
5     post(r : r <= x)       // a postcondition assertion
6                             // 'r' names the result object of 'func'
7 {
8     // observable checkpoint (P1494) right before evaluating the assert-argument
9     contract_assert(x <= 1); // an assertion statement, e.g. check invariants
10    // observable checkpoint right after returning from the handler in 'observe' semantics
11    return x + 2;
12 }
13
14 void call_func() {
15     func(0); // no contract violation
16     func(1); // violates precondition assertion of 'func'
17     func(2); // violates postcondition assertion of 'func'
18     func(3); // violates assertion statement within 'func'
19     func(4); // no contract violation
20 }
21
22 // -----
23
24 // in global namespace, global module, invisible to users
25
26 // called on any contract violation
27 // system-provided or optionally user-replaceable
28
29 void handle_contract_violation(std::contracts::contract_violation cvo) {
30     // handle contract violation according to
31     // - the evaluation semantics as indicated in 'cvo'
32     // - the rest of the info in 'cvo'
33 }

```



CONTRACT EVALUATION SEMANTICS

Semantics	check condition	fail → invoke handler	terminate
ignore (zero overhead)	✗	✗	✗
observe	✓	✓ ¹	✗
enforce	✓	✓	✓ ²
quick_enforce	✓	✗	✓ ³

¹ runtime: proceed after calling the handler / compile-time: diagnostic

² runtime: std::abort / compile-time: program is ill-formed

³ runtime: terminate abstract machine execution (i.e. no more user code)
compile-time: program is ill-formed



My takeaway

Use constant-evaluation to detect UBs

This talk reinforced a strong trend I've observed across Meeting C++ 2024: the community is actively **challenging outdated narratives about C++ being "unsafe by nature"**. Every new standard brings meaningful safety features, better diagnostics, and more predictable execution models.

🔗 I'm particularly excited about **contracts**. I want to start integrating them into my daily work as soon as tooling and compilers stabilize - it's exactly the kind of **self-documenting, correctness-driven mechanism** I've been waiting for, just like **concepts** before in another way (*like for constrained type-erasures / TEPS*).

I'm also eager to learn more about **STL hardening**, because I suspect enabling it will reveal lots of subtle bugs hidden in many existing codebases: bugs that today manifest only as "mysterious UB", and which are too often worked-around with dirty/questionable hot-fixes.

In truth, I almost have a kind of PTSD when it comes to undefined behavior 🤖.

Years ago - back when I was a junior developer -, I was involved in a project where a team spent nearly a full year chasing one particularly nasty UB issue (involving memory, concurrency, and AFAIR, some [SIOF](#)). It delayed delivery, ramped up pressure on everyone, and left managers baffled - pushing for quick, superficial fixes instead of a real investigation 🧐. Over and over again.

There is a French expression that fits such a situation: "*confondre vitesse et précipitation*" (literally: mistaking haste for effectiveness).

By the time a proper investigation was finally allowed and the full scope of the problem became clear, costs were already high and the organization's credibility with the client was seriously at risk. At the time, I was still a junior developer, with only a limited understanding of the C++ standard. But that is a story for another day.

Seeing the direction C++ is currently taking when it comes to safety - contracts, profiles, hardened libraries, expanded constant evaluation support - makes me optimistic.

Far from being an old unsafe language - *as the gossips and the ignorant says* -, C++ is steadily equipping developers with better tools to **write safe, robust, modern software**.

The beast keeps evolving to meet the need of the industry, and I can't wait to see what's coming next. This talk made that trajectory toward safety clearer than ever, and increased my appetite.

Raymi Klingers - 25+ Years of pathfinding problems with C++

 Video,  [slidedeck](#)

Raymi's talk explored more than two decades of pathfinding challenges in the [Age of Empires](#) franchise (*among the games that most shaped my early interest in real-time strategy as a child*), from the early [Ensemble Studios](#) codebases to the modern remasters developed by [Forgotten Empires](#).

Content

It was a deep dive into how [pathfinding](#) systems evolve over time, especially when dealing with legacy code, performance constraints, and increasing community expectations. The talk compared different approaches used in major RTS/MOBA titles (AoE2, AoE3 DE, Age of Mythology, [StarCraft II](#) 🐼, [League of Legends](#)) and highlighted how each game tackles core issues such as:

- pushing vs. non-pushing units
- bumping vs. non-bumping movement
- formation handling
- glitches like units clipping through walls
- overall pathfinding accuracy vs. performance tradeoffs

Raymi walked through the multi-year process of fixing and improving AoE pathfinding:

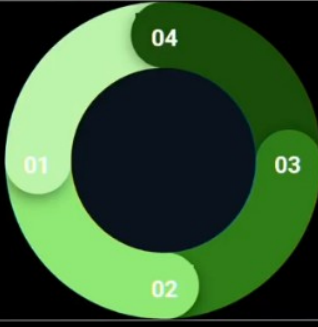
- intensive fuzz-testing to expose subtle bugs
- iterative bugfixing while ensuring no regressions
- dealing with floating-point precision loss (and adopting fixed-point to mitigate it)
- challenges with convex hull computations and complexity constraints
- the costliness of ray-tracing-based approaches

It was also framed in the context of remastering/modernizing old codebases - balancing "don't break the game the community loves" vs. "fix the long-standing problems players always complained about."

Meeting C++ 2025



Solutions: The Path to Robustness



1. Run Game

Lots of AI's spamming
literally millions of paths

2. Find Bug

Algorithm failed
Weird path
Unit stuck

3. Fix Bug

Changes to the algorithm
to fix the broken path

4. Run Suite

Check all previous cases
whether they still work



My takeaway

Even though this talk was engaging - especially because I love simulations and game mechanics - it felt a bit rushed and dense to me (*despite tons of cool slides tho !*), probably due to time constraints or stress. As a result, I couldn't fully grasp the nuances of the thoughts, decision process and algorithms; but the development journey was still fascinating.

Pathfinding has always been a captivating topic to me: I've built many small simulations and games on my spare time, but I've often avoided using - thus, implementing - pathfinding because I felt underprepared, worried about designing something both correct and performant. Still, the talk reinforced how much I want to learn more about it.

It also made me reflect on how pathfinding shares conceptual similarities (even if loosely) with collision detection engines: spatial reasoning, constraints, approximations, and the "classic" tradeoff between accuracy and performance.

This talk definitely encouraged me to dig deeper into:

- How large-scale games keep pathfinding predictable, stable, and maintainable
- The tradeoffs between precision and speed
- Historical/classic vs. modern pathfinding techniques
- How to make path-finding efficient yet testable
- Integrals vs. floating-point usage and strategies in performance-critical systems

This is the fourth talk that has sparked/rekindled my urge to build new simulations or games. Resisting the temptation is getting harder each time 🤖.

📅 11/08/2025

👤 **Klaus Iglberger** - The Real Problem of C++

 Video,  sliddeck

Klaus opened by addressing a harsh reality: major institutions such as the [NSA](#), the [White House / ONCD](#) (*source not available anymore*), and the European Union (via the [Cyber Resilience Act](#)) have publicly claimed that C++ is not appropriate for developing safe softwares.

Journalists amplified this message - often **conflating C and C++ under the misleading "C/C++" label** - feeding an increasingly **widespread belief** that C++ is an **unreliable and outdated** language.

For the record, C and C++ are respectively rated 2nd and 4th most used programming languages according to the [TIOBE index](#) in Jan. 2026.

Klaus's position is clear: **C++ is unsafe ... if you are still writing C++ like back in 1998.**

The real issue is not the language itself, but the mindset of many developers who do not leverage the modern features and tools introduced over the last ~30 years.

Safety vs. performance: an historical tradeoff

"C++ does not give you performance; it gives you control over performance.", Chandler Carruth.

C++ always leaned toward performance when design tradeoffs had to be made.

But modern C++ allows us to build safety on top of performance, while the other way around is not something intrinsically doable.

🔗 We - as developers - must use that control wisely.

Bounds safety

*"If you want to improve code quality in your organization, I would say, take all your coding guideline and replace them with the one goal. That's how important I think this one goal is: **No raw loops**. This will make the biggest change in code quality with your organization", Sean Parent*

Klaus insists this is the biggest category of avoidable safety bugs. Raw loops are simply outdated, unsafe, thus should be avoided.

- Prefer `std::ranges` and standards algorithms. If you are stuck with pre-C++20 for some reasons, then use the [range-v3](#) library.
- The key issue here is not a language problem: it's **resistance/reluctance to changing habits**.

Some peoples may have common misconceptions about using `ranges` over index-based for-loops: *"it's difficult, it's slow"*.

They are **wrong**: it is not, and that's what I call **pragmatic simplicity**. From your perspective, which of the following code is the most understandable for a non-developer ?

```
// C++98 style
std::vector<int> result;
for (int i = 1; i <= 10; ++i){
    if (i % 2 == 0){
        result.push_back(i * i);
    }
}
```

```
std::cout << "result: [";
for (int i = 0; i < result.size(); ++i)
    std::cout << result[i] << (i == result.size() - 1 ? "" : ", ");
std::cout << "]\n";
```

```
// C++23 style
auto view =
    std::views::iota(1, 11)
| std::views::filter(is_even)
| std::views::transform(to_square)
;
std::println("result: {}", view);
```

See the [demonstration here](#).

☹️ I notice such a resistance a lot when training developers, but ultimately they all come to understand that **pragmatic simplicity is just better than pattern-recognition**, for multiple reasons: less code, less bugs, easier-to-write and easier-to-maintain.

Undefined behaviors: the silent killers

As Daniela Engert mentioned in a previous talk (*see the conference previous day, above*), UBs appear ~90 times in the C++ standard, but it exists for a reason: it enables aggressive optimizations.

"Evaluation of a constant expression ([expr.const]) never exhibits behavior explicitly specified as undefined in [intro](#) through [cpp](#)."

C++ standard, [3.65 \[defns.undefined\]](#) undefined behavior

However, modern C++ gives tools to catch UB early:

- Undefined behaviors are not allowed at compile-time.
- Make as much code as possible `constexpr`.
- Use `constexpr` for tests: `static_assert(test())`.
- Test both compile-time and runtime paths.

🔗 As `constexpr` support expands each standard cycle, the space for UB keeps shrinking.

Type safety through strong types

Many bugs come from:

- Swapped function arguments
- Implicit conversions
- Narrowing conversions

Strong types solve this elegantly:

- [explicit constructors](#)
- type constraints (`std::same_as`)

- libraries like [rollbear/strong_type](#), [foonathan/type_safe](#), and [mp-units](#).

Same performance, significantly safer. That's what such zero-cost abstractions are all about.

Lifetime safety: value semantics over inheritance

Classic/legacy dogmatic [OOP](#) design in C++ leads to:

- pointer-heavy code
- dynamic allocations
- virtual calls / vtables
- complex lifetime management

Klaus emphasized that value semantics should be our default, which echoes with his previous talks like [C++ Design Patterns - The Most Common Misconceptions \(2 of N\) - CppCon 2024](#).

For instance, `std::variant` can replace entire inheritance hierarchies:

- no base classes
- no raw pointers
- no virtual dispatch
- fewer allocations
- safer code, simpler lifetime management

This captures the spirit of the "modern C++" mindset.

☞ But it's worth adding a nuance: the goal is **not** to replace **one blindly followed dogma** with another.

Speaking of value semantic, I noticed that C++26 comes with [std::polymorphic](#) and [std::indirect](#) to help promote value semantic.

See [p3019 - indirect and polymorphic: Vocabulary Types for Composite Class Design](#)

Conclusion: the real problem is a people problem

"No matter what the problem is, it's always a people problem." - Gerald M. Weinberg

Klaus closed with one of the strongest messages of the whole conference:

- Millions of C++ developers don't keep up with modern practices.
- Many still write 1990s-style C++ and then blame the language.
- Meanwhile, misinformation spreads loudly on social media.

His point was not to shame developers, but to remind the experts - us - to teach, mentor, write, and correct misconceptions.

☞ If we don't, loud voices will drive the narrative, not factual ones.

My Personal Opinion

"Some people prefer to learn a new programming language for several years instead of spending a few weeks to properly learn their own programming language."

(Klaus Iglberger)



The Expert's Opinion

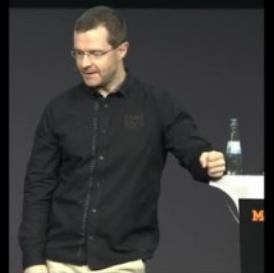


"If you want to improve code quality in your organization, I would say, take all your coding guidelines and replace them with the one goal. That's how important I think this one goal is:

No Raw Loops.

This will make the biggest change in code quality within your organization."

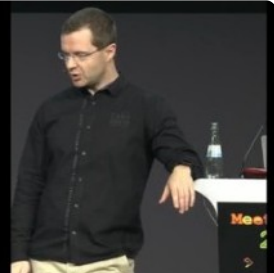
(Sean Parent, C++ Seasoning, Going Native 2013)



The Current State of the Discussion

"Some people prefer to search for programming errors for a few days instead of spending a few seconds to add a helpful line of code."

(Klaus Iglberger)



Meeting C++ 2025
woven by TOYOTA

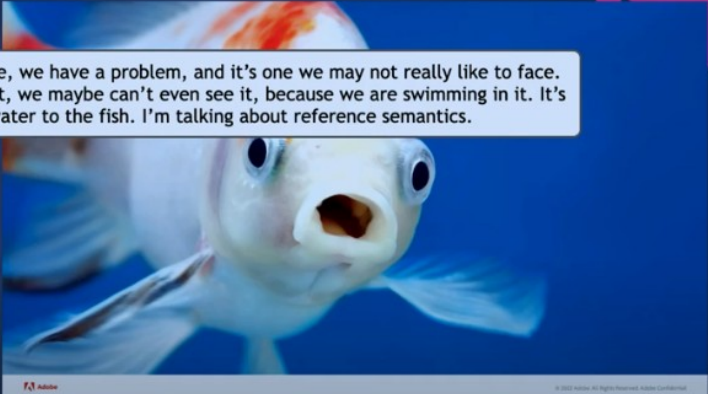

Cppcon 2022
September 12th-16th

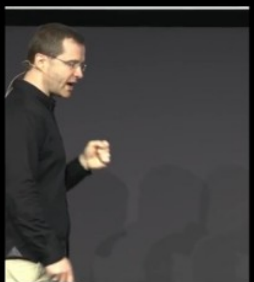


Dave Abrahams

Values: Safety, Regularity, Independence, and the Future of Programming

People, we have a problem, and it's one we may not really like to face. In fact, we maybe can't even see it, because we are swimming in it. It's like water to the fish. I'm talking about reference semantics.

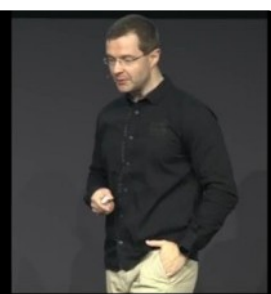




Conclusion



*"If there were 90-98% fewer C++ type/bounds/initialization/lifetime vulnerabilities we wouldn't be having this discussion."
(Herb Sutter)*



My takeaway

I've always admired Klaus's teaching style: deeply pragmatic, calm, rational, and laser-focused on **facts over dogmas**.

He questions sacred cows, re-evaluates assumptions, and isn't afraid to challenge outdated habits.

👉 That resonates strongly with my own philosophy as a trainer: break beliefs, fight misconceptions, and replace them with practical, evidence-based reasoning.

This talk felt perfectly aligned with the overall implicit "safety" theme of this year's conference. It tackled the elephant in the room head-on by naming the NSA, the White House, and the EU's "C++ is *unsafe*" claims. Honestly, **I'm fed up (no pun intended) with this narrative**, that is leading to some kind of a panick feeling among lots of my peers, but also managers and tech-leads.

😏 And especially when it's fueled by shallow takes, buzzwords, and biased/bad/malicious analogies.

This strongly echoes with the amazing MeetingCpp 2024 closing keynote "[Collective Amnesia ?](#)", from [Peter Sommerlad](#), in which he warned the audience against trend-driven/cargo-cult decision-makers, as well as simplistic solutions, ignorance and lack of pragmatism in general.

Klaus reminded me that safety in C++ is not a pipe dream. It's achievable today if we adopt modern techniques:

- ranges over raw loops
- strong types everywhere
- `constexpr` and `constexpr` to eliminate UBs
- value semantics as a default
- variants over inheritance
- using tools, sanitizers, static analyzers, [guidelines](#).

His talk strengthened my belief that the real issue is not the language - it's the collective mindset. And as someone who keeps learning, attending conferences, reading papers, and teaching, I'm part of the group that must mentor others.

If we - *those who stay up-to-date* - share our knowledge, challenge dogma, and counter misinformation, the "C++ is *unsafe*" rhetoric will fade. If we don't, we leave the floor to uninformed opinions.

This talk rekindled my motivation to teach contemporain, safe C++; to fight bullshit wherever it comes from - old university teachings, trends, buzzwords, or social-media spread myths - and to push for a community that **values evidence and pragmatism over beliefs and dogmas**.

☞ From my perspective, the C++ #1 strength is its [standard](#) - the **ultimate, formal source of truth**. However, most C++ developers don't even know it exists, and favor unreliable sources such as LLMs, forums, SO, etc.

Klaus delivered one of the clearest and most impactful messages of the entire conference, and I'm genuinely grateful for it.

He concluded with a call to action:

- We must **keep improving the language** so that writing **safe code becomes easier**.
- We must **communicate and educate** more effectively about **how to write C++ properly**.

Additional sources

- [The Case for Memory Safe Roadmaps - Why Both C-Suite Executives and Technical Experts Need to Take Memory Safe Coding Seriously](#), December 2023, CISA | NSA | FBI | ASD's ACSC | CCCS | NCSC-UK | NCSC-NZ | CERT-NZ.
- [Software Memory Safety](#), April 2023, National Security Agency | Cybersecurity Information Sheet.
- [Peter Sommerlad - "Collective Amnesia ?" - Meeting Cpp 2024](#)

👤 **Roth Michaels** - How to become obsolete: a guide to software engineering mentorship

📺 [Video](#), 📄 [slidedeck](#)

Roth framed his talk around a question we - *experienced engineers* - all eventually face: How to lead effectively, share knowledge, yet still avoid being the bottleneck ?

The talk was heavily inspired by Meeting C++ 2024 themes ([Titus Winters on "Fear in C++"](#), [Kate Gregory on "Aging Developers"](#)), and tied to Roth own experience at [Native Instruments](#).

The central metaphor was [free jazz](#) - a style built on improvisation, shared leadership, and trust, but also breaking conventions, to get rids of limitations in order to promote creativity.

Why become obsolete ?

The goal of a senior engineer or tech-lead should not be to remain indispensable forever. If the team relies on you for every decision, you will never move on to new opportunities, and the team will never grow.

☞ In free jazz terms: you're not the conductor - you're another musician in the ensemble.

Becoming "obsolete" means that:

- You've successfully **shared leadership**
- The team can run without you
- You've built **resilient structures for decision-making**

And when that happens, you gain the space to breathe and ask to yourself : "What's the next important thing I can achieve/work on now ?"

Shared technical leadership

Roth emphasized **pair-leadership**, similar to pair-programming but applied to ownership of systems, architectures, and projects directions.

☞ This promotes resilience and stability among the team. Two leaders **share responsibilities**:

- Project oversight
- Architectural decisions
- Reviewing code
- Offering technical expertise

This ensures that if one person is unavailable at a moment (sick or on vacation) then **nothing falls apart**. Knowledge redundancy is a leadership achievement.

Mentorship must be part of the **onboarding process**: new hires should pair with experienced peoples, to be shown:

- What the **project's architecture** is
- How the **codebase is organized**
- What the **guidelines, principles** and **best-practices** are
- Be taught thought **processes** - not just tasks (**decision-making**, weightening **trade-offs**, etc.)

Identifying and developing leaders

☞ Roth insisted that true leadership often lies in reluctant leaders, not the loudest voices.

Technical leadership should not be exclusive to extroverts or natural talkers, but instead:

- Find subject-matter experts
- Observe quiet but thoughtful engineers
- Rotate projects leaderships
- Every project can (and should) have a different project lead.

Meeting C++ 2025



Identifying leaders



- Obvious tech leads
- Non-obvious or reluctant leaders
- Find subject matter experts
- Look for opportunities to share leadership

Practical mentorship techniques

Some key practices Roth highlighted:

Sharing knowledge

- Wikis usually fail: too long to write, rarely read -> not cost-effective
- **Answer questions publicly**, not in private channels
- Organize live discussions for design decisions -> collective intelligence
- Direct questions to subject-matter experts
- Provide references, not only answers -> formal sources of truth

Talking through engineering problems, asking them

- What is the actual problem?
- What is your thinking?
- Why is this a good - or bad - idea?

💡 And **be transparent about the strength of your ideas**: some are strong, some are half-baked.

👉 Leaders' opinions carry lots of weight; they must communicate uncertainty clearly.

Pair-debugging is an incredibly powerful teaching tool

- Ask "show it to me in the debugger"
- Place breakpoints, watchpoints
- Explore the state together

💡 It lets you teach not only the solution, but your whole investigation and decision-making process.

Code-reviews: differentiate types of comments

- Neat suggestions
- Educational remarks
- Blockers / must-fix

⚠ He also warned about "LGTM anxiety": is it genuine approval or avoidance ? When code is complex (TMP, concurrency, etc.), do live walkthroughs, but remain aware of the bias risk (e.g false sense of understanding).

Encourage psychological safety

Roth advocates **bragging about mistakes**, reminding me of the "**monkey of the week**" practice, which is encouraging your ICs to laugh about their own mistake - in a public channel, during sprint retrospective, etc.

🔑 Teams need a culture where failure is acceptable and even encouraged as a learning mechanism.

💡 Leadership = vulnerability + openness + candor.

Q&A

I had the opportunity to ask about presenting complex PRs (TMP, concurrency, multi-threading, etc.) without biasing reviewers.

Roth's answer was straightforward:

- A reviewer must understand **every** part of the PR.
- If they don't, they should ask and learn.
- If people don't understand a pattern or technics, then write/share minimal examples to teach it to others.

As a lead developer, I have often seen contributors schedule "*PR presentation*" sessions with me. While well-intentioned, these meetings can - *from my perspective/experience* - create a **false sense of shared understanding** and **introduce additional pressure** to approve the change quickly, often in order to meet deadlines or delivery milestones.

I eventually decided to discourage such meetings altogether, as they too often left me with the uncomfortable impression that I had agreed to a change without fully scrutinizing it. In several cases, this resulted in bug-prone and/or hard-to-maintain code making its way into the codebase-despite an apparent consensus reached during the discussion.

These meetings were replaced with design and/or implementation strategy sign-offs, during which developers present their intended approach (for example, how they plan to design a feature or resolve an issue). Once the pull request is submitted, a follow-up discussion takes place to assess the result and to analyze any delta between the original intent/promise and the final result.

My Takeaway

This talk hit close to home for me, because mentorship and technical leadership have become a growing part of my own work.

I loved how humble and empathic Roth was, and how his analogy with free jazz was relevant: guiding without dictating, structuring without suppressing creativity, and enabling others to improvise confidently.

His emphasis on becoming "obsolete" is something I strongly believe in: leadership isn't about centralizing authority, but about distributing it so the team can thrive even without you.

☞ Unfortunately, this ideal is often contrasted sharply by what we see in practice.

Too many leaders - *especially managers* - lacking technical understanding still impose ill-informed decisions with little or no justification.

This kind of unilateral, arbitrary behavior echoes the textbook definition **fascism**: an authoritarian, arbitrary, and imposing attitude exerted by an individual over a group.

☞ Leadership and respect should not be asserted, but **earned** by demonstrating kindness, guidance, and mentorship.

His point about **reluctant leaders** resonated particularly deeply. In many teams, the loudest voices dominate by default, while some of the most thoughtful, reliable, and technically insightful engineers are quieter. Identifying and nurturing them is a huge multiplier for team health.

☞ Extroverts who like to brag and to tell stories are too often favored, to the detriment of competent introverts.

💡 Related to this, I recently had the opportunity to discuss what could be described as *courtier/courtesan/sycophant* roles: ambitious individuals who are highly articulate, socially adept, and well-regarded by top-level management, yet whose influence is primarily **performative rather than grounded in technical reality**.

By **overpromising outcomes, underestimating complexity**, or presenting an **overly optimistic view of feasibility and timelines**, they create a **false sense of shared understanding** at the management level.

This, in turn, translates into **unrealistic expectations**, compressed deadlines, and **sustained pressure** on engineering teams.

Despite being frequently rewarded and promoted for their apparent effectiveness, such behavior is ultimately detrimental to project delivery, team morale, and long-term trust.

I also appreciated his very pragmatic mentoring techniques: pair-debugging, public Q&A/AMAs, tackle uncertainty, and using discussions as a way to transmit thinking patterns, not just decisions.

From my perspective, building a **shared/common understanding** of a project's vision/goal, philosophy, architecture, but also tradeoffs, milestones, and constraints is a key to achieving success.

💡 To use nautical metaphor, a software team is much like a ship's crew.

When the crew does not know where the vessel is headed, does not trust the captain, or is not given a clear course - its destination, waypoints, and the reasons behind the journey - even the most skilled sailors will struggle. Effort becomes fragmented, initiative fades, and progress turns erratic.

Conversely, when the destination is explicit, the route is understood, and leadership is trusted, then each member of the crew can act with confidence and autonomy. In such conditions, individual expertise compounds rather than conflicts, and the ship advances steadily toward its goal, even in rough seas.

Overall, the talk was refreshing, human, and deeply practical: a reminder that leadership is less about the cult of personality, and more about **creating ecosystems where others can grow**, take ownership, and eventually replace you.

👤 **Kerstin Keller** - Why managing C++ dependencies is hard (and what to do about it)

📺 Video, 📄 slidedeck

With this talk, [Kerstin Keller \(AUTOMOVIO\)](#) discussed why C++ dependency management is uniquely difficult, and how modern tools - and [Conan](#) in particular - can help.

This is a nice addition to her talk back at Meeting Cpp 2024 [Clean CMake for C++ \(library\) developers](#).

Dependency management matters because most - *if not, all* - projects rely on many external components, but keeping everything portable, compatible and scalable is a non-trivial challenge.

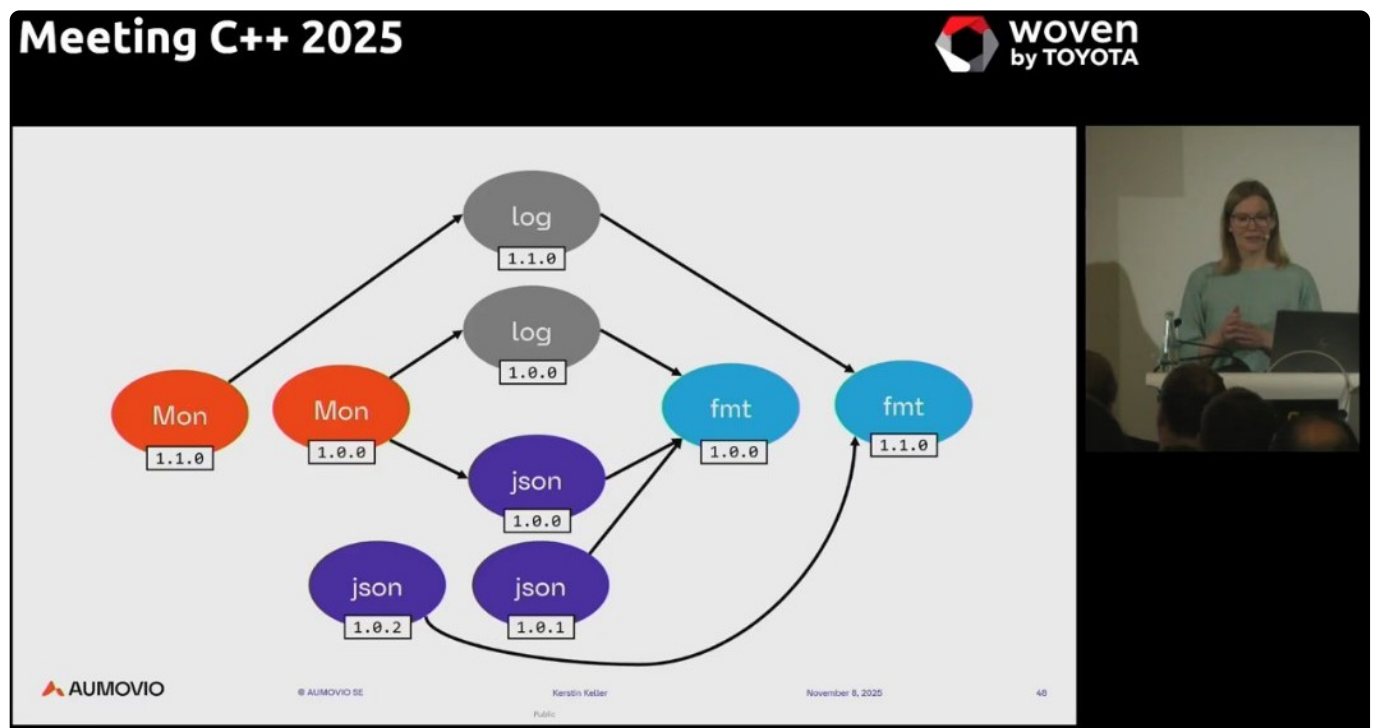
She began by explaining that dependencies are usually extracted outside the main repository: projects combining their own code with the full source of all external libraries would simply be too big.

What are dependencies & what is dependency management ?

A **dependency** is any external component (library, framework, module, service...) required to compile or run.

Dependency management is about orchestrating these components across versions, environments, and build configurations.

Semantic **versioning** (*major.minor.patch*) is a contract between library authors and users, and reproducible builds require consistent source, environment, and instructions.



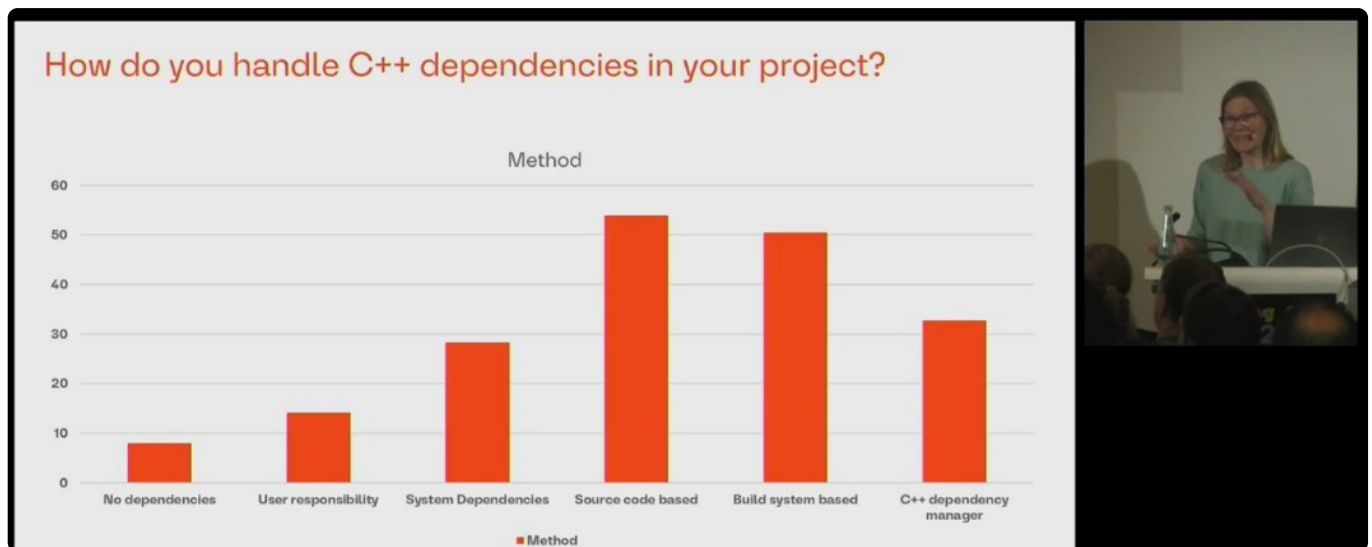
How developers handle dependencies nowadays ?

- Source-based vendoring (copying the dependency into the repo) -> not scalable, too heavy, hard to update
- Build-system-based (like [CMake's FetchContent](#))
- Dependency managers ([Conan](#), [vcpkg](#), [Hunter](#), [Biicode](#), etc. that are still rarely used in C++ projects)
- System packages (Linux distros, like [apt](#), [yum](#), etc.)

💡 Conan feels great

Dependency managers help solve multi-version graphs, transitive dependencies, and mismatched constraints.

💡 Pin major versions, avoid broad ranges.



Why C++ dependencies management is unique and hard ?

Three main challenges:

- **Binary compatibility:** When is an existing package safe to reuse ?
Tools compute a package ID hash (compiler, options, build settings) to detect **ABI compatibility**.
- **No standard build system:** C++ projects use [CMake](#), [Meson](#), [Bazel](#), custom/hands-written [Makefiles](#), etc.
[Conan integration](#) supports most.
- **No standard package format:** Different ecosystems lack a shared way to describe installable artifacts.
👉 The proposed solution: [CPS \(Common Package Specification\)](#) - a JSON-based declarative description of how packages may **easily** consume each others, from a **build** perspective.

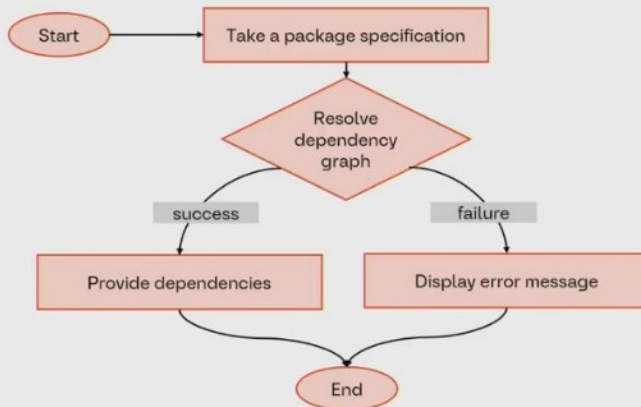
💡 Both [CMake](#) and [Meson](#) are experimenting with [CPS](#) support.

Meeting C++ 2025



C++ dependency management challenges

The tasks of a dependency manager



AUMOVIO

© AUMOVIO SE

Kerstin Keller

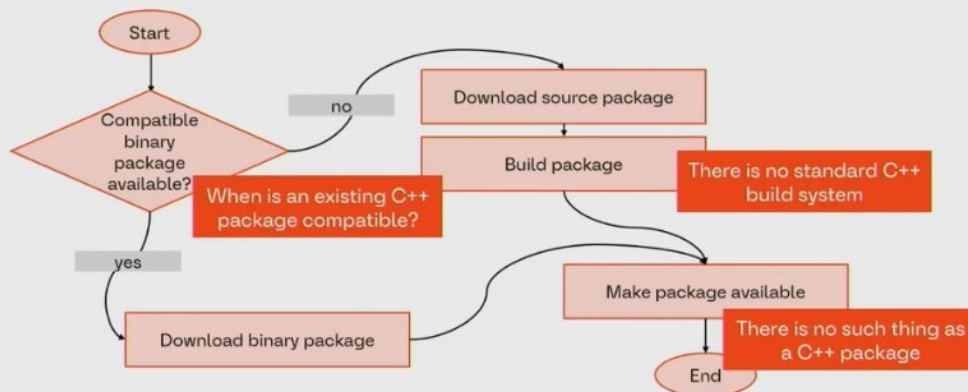
November 8, 2025

56



C++ dependency management challenges

Providing dependencies



Inputs from the Q&A

- [Conan](#) integrates well with [JFrog Artifactory](#).
- [CMake Package Manager \(CPM\)](#) still operates at the build-source level, not binary level.
- [Conan](#) vs [vcpkg](#): [Conan](#) supports binary packages out of the box, reducing build times for large dependency graphs.
- [Conan](#) profiles can live in a central [Git](#) repository.
- [Conan](#) can generate [CMake](#) toolchain files.

My Takeaway

In a nutshell:

- Dependencies form complex **graphs**; dependency managers automate retrieval and updates.
- **Lockfiles** are essential for reproducible builds.
- Some may complain about the C++ ecosystem lacking a standardized packaging and build tooling, which might make dependency managers harder to implement.

- [Common Package Specification \(CPS\)](#) is a promising cross-tooling package description format the community should support.
- 📌 Use [Conan](#).

I spend a lot of time building resilient/reliable, portable, modular C++ systems (*including some with ROS !*), and dependency management is often a central pain point.

From my perspective, this talk evangelized strongly about "*just use Conan*", but this seems to be a strong and realistic tool to tackle dependency-management related issues nowadays (*ABI compatibility, package format standardisation, and build systems homogeneity, etc.*).

Honestly, I mostly use [CMake's FetchContent](#). Sometime with a front built on top to promote consistency and a better API: [CPM](#) or a homemade one.

And it works not too bad: sure, fetching takes time, but then most of my projects take from a fistful of seconds up to 2 minutes to build, even larger ones. Especially because nowadays, most C++ libraries are - or offer some variants as (like [fmt](#) with [FMT_HEADER_ONLY](#)) - header-only.

📌 Although, I like A LOT the idea of just pressing the [build](#) button in my IDE, and let the project(s) take care of its dependency tree all by itself.

Attempting to use, build or contribute to some GitHub projects still often devolves into an iterative cycle of missing dependencies, manual installation, and repeated build failures.

From my experience, still too many companies often manage dependencies with a mix of:

- Embbed everything in a [Docker](#) (dev)container using the OS package-managers ([apt](#), [yum](#), etc.).
- [git](#) submodules
- copy-pasted files directly in the source tree 🌀
- CMake [FetchContent](#) and/or [file\(DOWNLOAD ...\)](#).

The [CPS](#) part was especially insightful, as I was not aware of such an initiative: C++ desperately needs a unified package description, and hearing that [CMake](#) and [Meson](#) are converging on something concrete is encouraging.

Overall, I came out with a clearer overall picture of why dependency managers in C++ feel so rough, why I often struggle with it, and how the ecosystem might evolve in a - *hopefully, near* - future.

It'll also make sure to give [Conan](#) a decent try when I'll have the opportunity !

Additional resources

- [A drop in replacement for pkg-config/pkgconf using cps files](#)
- [CppCon 2023 - Libraries: A First Step Toward Standard C++ Dependency Management - Bret Brown & Bill Hoffman](#)
- [CppCon 2023 - A Common Package Specification: Getting Build Tools to Talk to Each Other - Lessons Learned](#)

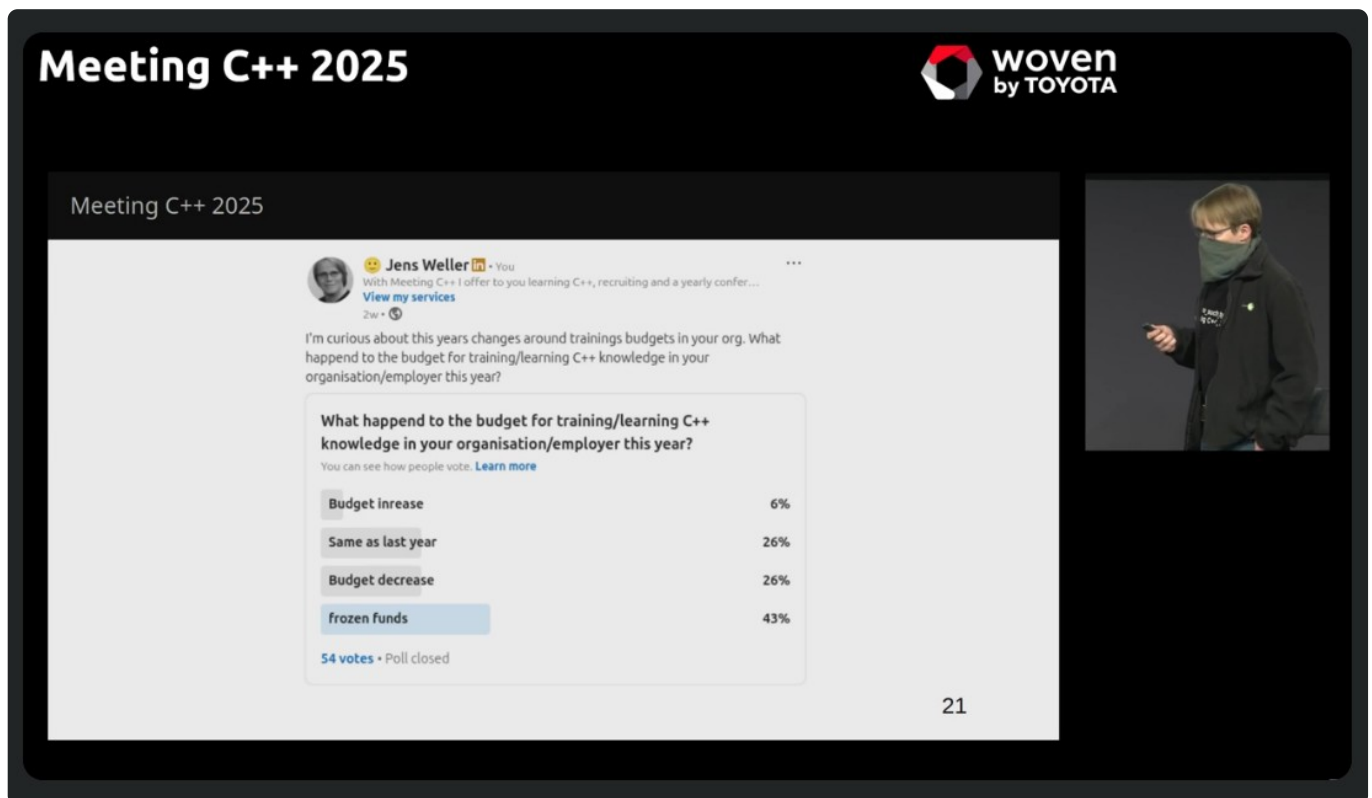
👤 **Jens Weller** - Meeting C++ update

📺 Video, 📄 [sliddeck](#)

Like every year, Jens Weller shared a transparent update on the state of **Meeting C++**, and the broader conference ecosystem.

While the conference reaches nearly **100000 online visitors** compared to about **230 onsite attendees**, 2025 has been economically challenging: frozen training budgets, fewer committed sponsors, and rising fixed costs - especially for hybrid events hosted in high-quality venues.

Despite this, Meeting C++ is still standing after **13 years**, this iteration being described by Jens as "*the best conference in the worst year*", though just barely breaking even. Jens was clear: **the conference must continue to evolve & adapt**, as running it the same way has become too risky.



Meeting C++ 2025

woven by TOYOTA

Meeting C++ 2025

Jens Weller • You
With Meeting C++ I offer to you learning C++, recruiting and a yearly confer...
[View my services](#)
2w • 🌐

I'm curious about this years changes around trainings budgets in your org. What happend to the budget for training/learning C++ knowledge in your organisation/employer this year?

What happend to the budget for training/learning C++ knowledge in your organisation/employer this year?

You can see how people vote. [Learn more](#)

Budget increase	6%
Same as last year	26%
Budget decrease	26%
frozen funds	43%

54 votes • Poll closed

21

What's next ?

Meeting C++ has already diversified significantly to stay sustainable: online shops ([hoodies and tshirts](#), and [calendars](#)), fund-raising initiatives, paid services (trainings, code reviews), live and pre-recorded talks, [job fairs](#), [newsletters](#), [blogs](#), [blogroll](#), and experiments with new online formats like AMAs, livestreams, and speaker interviews (check the [youtube channel](#)).

Jens also emphasized the foundational role of C++ user groups, which were key to launching Meeting C++ in the first place and remain central to its mission-supporting new groups, sharing their events, and connecting local C++ communities.

2026 conference

The next Meeting C++ conference is planned for November 26-28 2026, again in Berlin ([Andel Hotel](#)), with both onsite and online participation.

Jens encouraged attendees to support the conference early by buying tickets sooner in future years.

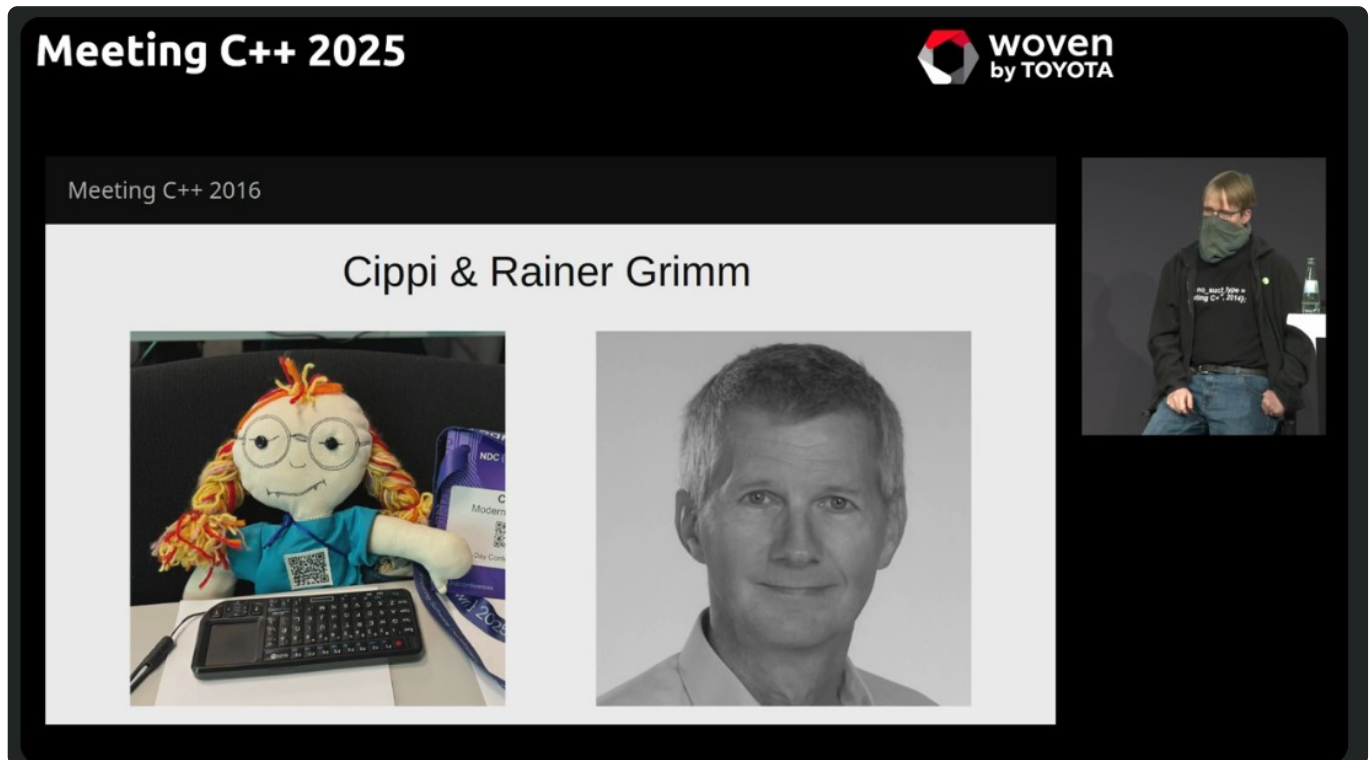
The update closed with heartfelt thanks to sponsors, volunteers, speakers, and attendees, and the C++ community.

Hommage to Rainer Grimm

The update ended with an unplanned, emotional homage paid to [Rainer Grimm](#), who recently [passed away](#) from [ALS \(amyotrophic lateral sclerosis\)](#), possibly linked to (long) Covid.

Jens also shared the symbolic journey of [Cippi](#) the puppet accompanying Klaus Iglberger to conferences around the world, keeping Rainer's memory alive within the C++ community.

👉 Go check [Cippi's blog](#) !



(I put this screenshot twice on purpose)

⚡ Lightning talks

⚡ James McNellis Our most teacherous adversary

📺 Video, 📄 slidedeck

James humorously reminded us that as C++ developers, we sometime over-trust compilers, which might break our code whenever the standard allows them to.

His example with `bool` illustrated how seemingly harmless types can behave unpredictably across privilege or ABI boundaries.

To my knowledge, `bool` suffers from an unstable, implementation-defined representation - as the C++ standard does not fix how it is represented in memory. It just may vary across compilers and platforms, from 1 byte as bit value (0 or 1), 1 byte (0 or non-zero, like 42), 4 bytes, packed or padded in structs, etc.

`bool` - integer type, capable of holding one of the two values: true or false. The value of `sizeof(bool)` is implementation defined and might differ from 1.

[cppreference](#)

👉 Takeaway: **never use bool in data structures that cross boundaries** like files, networks, or IPC.

Such a type can be quite pitfall - *especially for junior developers* -, and safer explicit fixed-width integer types

should be preferred.

Beyond the technical aspect, this once again demonstrates how important it is for us developers to **be well-aware of the standard**, rather than constructing a mental representation that may differ from reality; and for trainers/teachers to not over-simplify things.

💡 This brings to my mind A.Einstein's quote "[...] *the supreme goal of all theory is to make the irreducible basic elements as simple and as few as possible **without** having to surrender the adequate representation of a single datum of experience.*", often paraphrased as "*Everything should be made as simple as possible, but not simpler*".

From my perspective, that's what C++ is fundamentally about: **continuously expanding what's possible to achieve with code**.

Meeting C++ 2025



```
std::string boolToString(bool value)
{
    return value ? "true" : "false";
}

lea    rax, ["false"]
lea    rbx, ["true"]
test   rdi, rdi           ; rdi contains 'value'
cmovne rax, rbx           ; rax contains pointer to string
```



👉 Allow me a quick digression: people often complain that C++ has a steep learning curve (and I partly agree), using things like PRNGs as an example. Sure, Python's random module gives you a simple API - `dice_roll = random.randint(1, 12)` - and it works. But what's the seed? What distribution is being used? If you're simulating the sum of two dices, you'd probably prefer a Gaussian-like distribution rather than a uniform one.

Which is perfectly feasible in Python, but not without undermining the fallacious argument of simplicity.

Giving it a quite/naive quick shot (*disclaimer: I'm not a Python developer, not into maths*):

- [Python](#)
- [C++](#)

⚡ Rahel Natalie Engel - let them eat cake

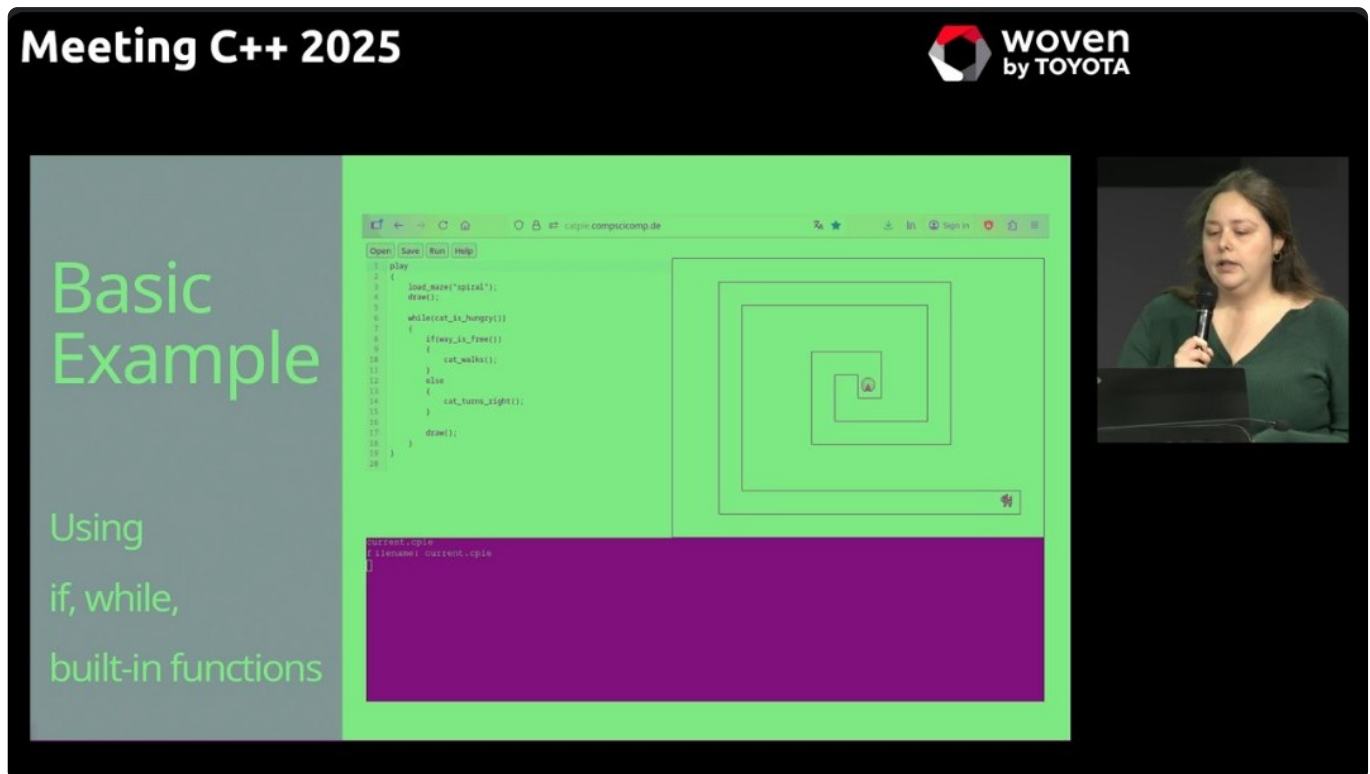
 [Video](#),  [slidedeck](#)

Rahel discussed diversity challenges in tech and shared lessons from university diversity and outreach programs.

Many beginners start with [visual programming](#) tools like [Scratch](#) due to time and accessibility constraints, but this often makes the transition to text-based programming difficult.

As a solution, she presented [catpie](#), an accessible hybrid visual-text tool meant to ease that transition and make early learning more accessible.

- [Conference paper, on ResearchGate](#)
- [Project's sources, on Gitlab](#)



💡 A thoughtful reminder that the tools **we choose for beginners often shape their long-term learning path.**

Making programming accessible & fun is crucial, but we must also ensure learners can gradually grow into full programming languages without unnecessary friction.

⚡ Robin Savenen Soderholm - vector to array: let the compiler turn heap into static storage

📺 Video, 📄 slidedeck

Robin showed how a dynamic, heap-based design used on a Raspberry Pi could be migrated to a much more constrained [Raspberry Pi Pico](#) by leaning on `constexpr` and `constexpr`.

By letting the compiler evaluate everything at compile time, a `std::vector`-like workflow effectively turns into static storage without runtime allocation, which meets the target platform constraints.

In a nutshell, he presented a simple yet effective `range_to_array` function, inspired by [Mikhail Svetkin's talk](#) "Harnessing `constexpr`: A Path to Safer C++".

- takes an `std::invocable` as parameter that return type meets the `std::range` requirement.
- the result type is an `std::array` which `std::ranges::range_value_t` is the `std::invoke_result_t` of the `invocable`, and size is the `range::size` of the `invoke_result_t`.
- copy elements generated by the invocable into the array

The implementation was something like:

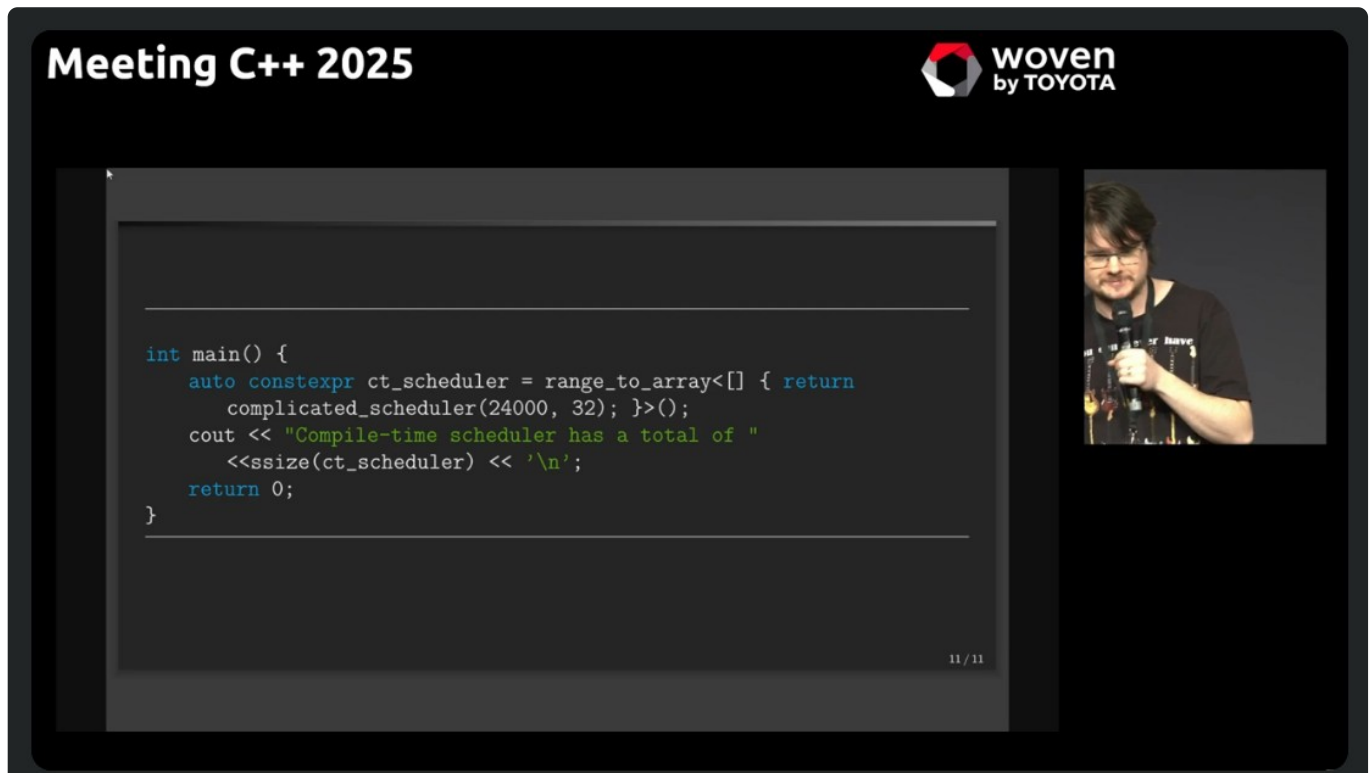
```
template <std::invocable auto generator>
    requires std::ranges::range<std::invoke_result_t<decltype(generator)>>
constexpr auto range_to_array() {
```



```

using generator_result_type = std::invoke_result_t<decltype(generator)>;
auto result = std::array<
    std::ranges::range_value_t<generator_result_type>,
    std::ranges::size(generator_result_type)
>{};
std::ranges::copy(
    generator(),
    std::ranges::begin(result)
);
return result;
}

```



⚡ Ganest Rengasamy - Meet Qt

📺 Video, 📄 slidedeck

Rather than explaining what the [Qt](#) is, this was a refreshing reintroduction that showed how the framework has evolved to keep up with users and modern industry needs.

The talk highlighted [Qt](#) as a broad, end-to-end platform - well beyond just [GUIs](#) - covering cross-platforms, embedded, desktop, mobile, backend integration, modern UI workflows ([Figma](#)), and even 3D assets.

Through an industrial vehicle use case, it showcased [Qt](#) as an end-to-end ecosystem: from backend services ingesting sensor data via [MQTT](#) to Qt-powered UIs designed in [Figma](#) and even rendering assets from [Blender](#).

I haven't really had the opportunity to use [Qt](#) since my school days (about 15 years ago), where it mostly meant - *as far as I remember* - GUIs and signal/slots. Back when I was freelancing, I tended to favor C# with [WPF/XAML](#) for GUI development, as the [WYSIWYG](#) experience felt more mature and better integrated to me at the time.

Seeing how much broader and more modern the ecosystem has become makes me genuinely curious to revisit Qt and explore what other powerful features it offers today to ease my developer needs.

Additional resources:

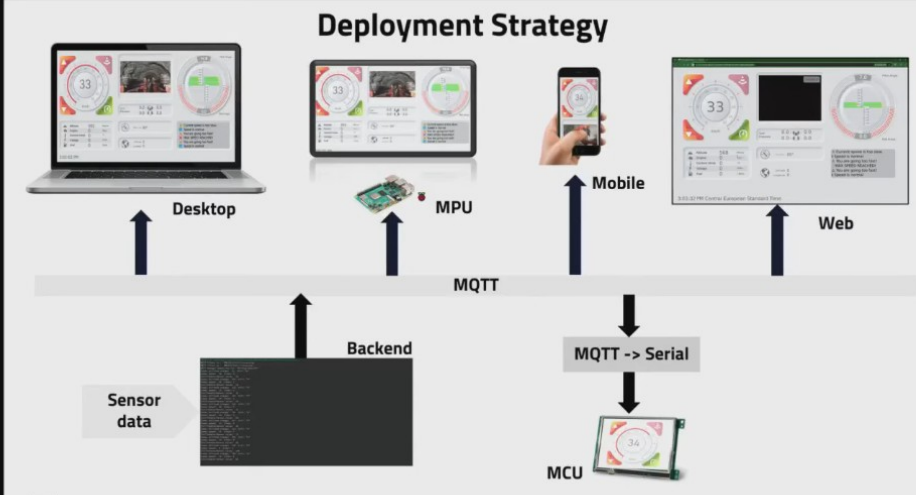
- [Qt: Exporting from Blender](#)
- [Qt: The Fastest Way to Bring Figma Designs to the Product](#)

Meeting C++ 2025

Meet Qt - Ganesh Rengasamy



Deployment Strategy



The diagram illustrates a deployment strategy for Qt. At the bottom, 'Sensor data' feeds into a 'Backend'. The 'Backend' connects to a 'MQTT' hub. From this hub, data is distributed to four client types: 'Desktop' (laptop), 'MPU' (microcontroller), 'Mobile' (smartphone), and 'Web' (tablet). Additionally, the 'MQTT' hub connects to a 'MQTT -> Serial' interface, which then connects to an 'MCU' (microcontroller unit).

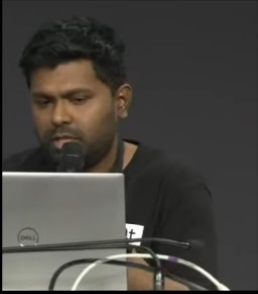
Qt Group

Confidential

© The Qt Company

8 November 2025

6



⚡ Hannah Lenk - Start teaching C++

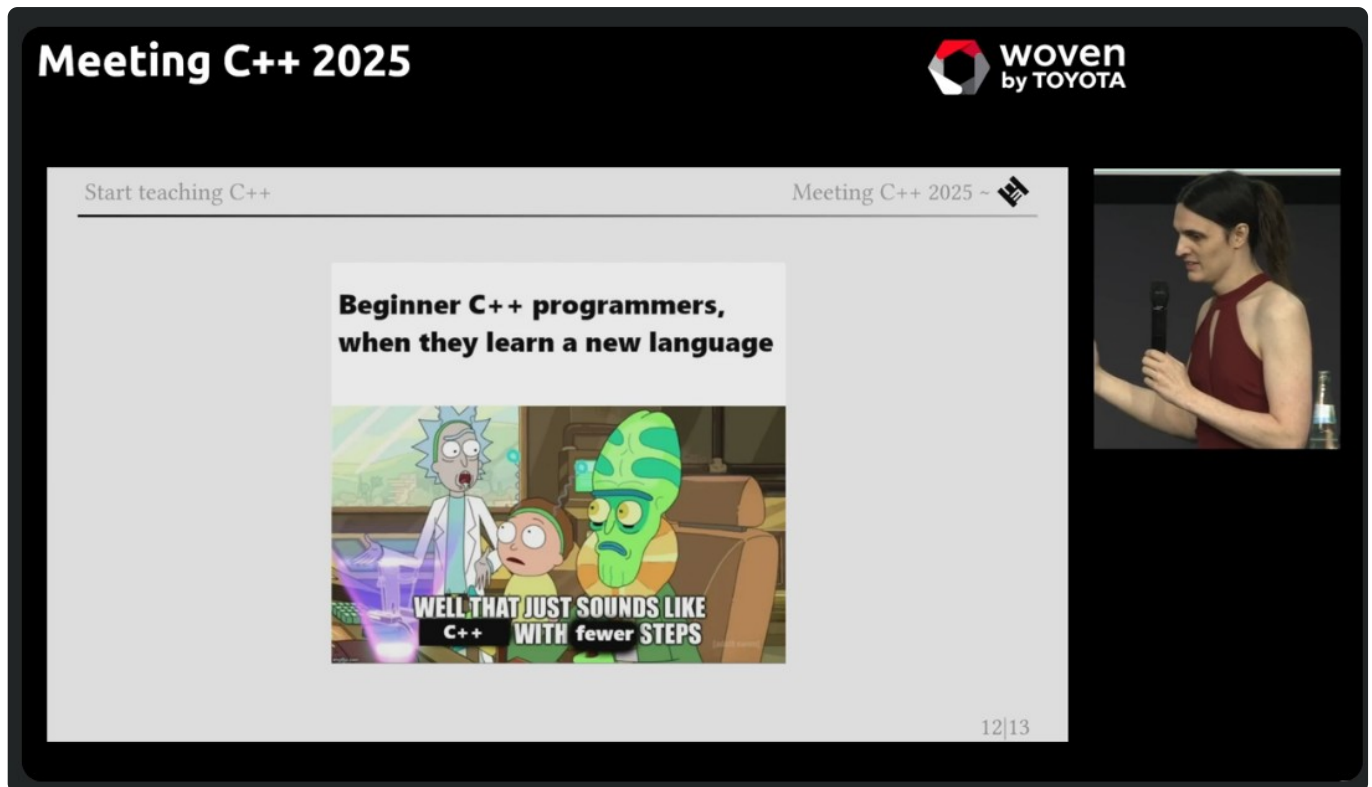
 Video,  [sliddeck](#)

Hannah argued that much of the criticism aimed at C++ stems from **outdated usage** and a **lack of teaching** rather than from the language itself. Echoing several talks at the conference (Klaus Iglberger's, Daniela Engert's), she challenged the idea that C++ is inherently complex or unsafe, reminding us that **simple does not mean familiar** (see [Pragmatic Simplicity - Actionable Guidelines To Tame Cpp Complexity - Vittorio Romeo - CppCon 2022](#)) and that C++ remains one of the most widely used languages today (top 4 in the [TIOBE index](#)).

👉 From my perspective, C++ doesn't have a language problem - it has a **teaching problem**.

If we want better C++ code and fewer misconceptions, we should **start teaching contemporain C++** to learners (beginners and professional developers), instead of letting **loud, uninformed voices dominate the narrative**.

Teachers have a huge responsibility here: they should stop teaching C++98 or that so-called "C/C++", and start teaching modern, simple, elegant and safe contemporain C++. C++ in 2026 feels like another language that C++98.



⚡ Jens Weller - binary trees

📺 Video, 📄 slidedeck

Jens shared a thought process that started with a simple question: what does the web - *including LLMs chatbots*- teach us today about trees in C++ ?

Unsurprisingly, most examples still revolve around pointer-based binary trees, often written in a C-with-classes style using raw pointers, `new/delete`, and little consideration for modern design; which results in inconvenient, error-prone APIs. With developers increasingly turning to AI assistants instead of search engines, **these patterns are now being amplified rather than challenged** - `ChatGPT`, `Grok`, `Gemini` and similar LLMs tools largely reproduce what they find online, including outdated or incomplete practices.

From there, the talk pivoted toward [data-oriented design](#).

Drawing inspiration from past keynotes Jens explored replacing pointers with indices stored in contiguous containers.

- [CppCon 2014: Mike Acton - "Data-Oriented Design and C++"](#)
- [CppCon 2025: Vittorio Romeo - More Speed & Simplicity: Practical Data-Oriented Design in C++](#)

Using indices instead of raw pointers sidesteps incomplete-type issues, enables optional-like semantics, and improves cache locality.

Initial benchmarks (*using [quick-bench](#)*) comparing naive pointer-based trees to index-based implementations showed significant performance differences (about x2). "Not very suprising" - as Jens concluded - yet always worth reminding.

Rather than claiming a definitive/absolute solution, Jens framed this as an exploration. He raised follow-up questions about fairness in benchmarking, allocator effects, and how much of the observed gain comes from cache-friendly layouts versus allocation strategies.

The talk closed by positioning trees as a fertile playground for revisiting long-assumed design choices in the light of modern hardware and language capabilities.

My takeaway

What started as a casual survey of "binary trees on the internet" quickly turned into **a lesson in intellectual hygiene**.

By showing how most online examples and LLMs-generated answers still promote pointer-heavy, legacy-style implementations, Jens highlighted a deeper issue: many developers, especially juniors, increasingly treat AI tools and forums as a primary source of truth rather than as starting points for critical thinking.

From my perspective, this creates a feedback loop.

🔗 LLMs largely reflect the datasets they were trained on, which means they tend to echo and amplify outdated practices. These patterns then get reused for learning, ramp-up, and ultimately copied verbatim into production code and pull requests.

The talk was a timely reminder that contemporary C++ requires discernment: tools can assist us, but responsibility for evaluating design choices - and updating our mental models - still firmly rests with us.

Meeting C++ 2025
Trees in C++ - Jens Weller

woven by TOYOTA

Using an index?

- An Index into a container is stable, if...
 - A non growing, preallocated container
 - By container design (e.g. `unordered_map` (collisions?!?))
 - Index into an allocator ("handle")

119

👤 **James McNellis** - Closing Keynote - A little introduction to control flow integrity

📺 [Video](#), 📄 [slidedeck](#)

Speaker: James McNellis, game engine security at ROBLOX About C++ being unsafe (laughs) like about half of the talks here this year.

[James McNellis](#) (Roblox) closed the conference with a deep yet tour of [control-flow integrity \(CFI\)](#), grounded in real-world exploitation techniques and modern defenses techs.

He started with a simple & common (and deliberately vulnerable) C example to illustrate how classic buffer overflows can lead to arbitrary code execution, especially when attackers can overwrite return addresses

without triggering immediate crashes.

From there, the talk walked through the historical evolution of CFI hacks mitigations: stack canaries, non-executable memory, and address space layout randomization. While effective, these defenses pushed attackers toward more sophisticated techniques such as [return-to-libc attacks](#) and [return-oriented programming \(ROP\)](#), where existing instructions - "gadgets" - are chained together to hijack execution without injecting new code.

This led into "CFI v2": stronger guarantees even in the presence of arbitrary read/write vulnerabilities. James explained how [shadow stacks](#) (notably Intel CET), [ARMs pointer authentication \(PAC\)](#), and newer mechanisms like [ARM's Guarded Control Stack \(GCS\)](#), comparing their security properties and tradeoffs.

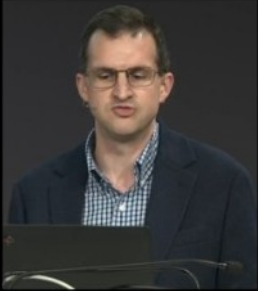
The second half of the talk addressed (no punne intended) - function pointers and virtual dispatch - covering [Microsoft's Control Flow Guard](#), [Clang CFI](#), and hardware-assisted mechanisms like [Intel's Indirect Branch Tracking \(IBT\)](#) and [ARM's Branch Target Identifier \(BTI\)](#), all designed to restrict execution to valid control-flow targets.

Meeting C++ 2025

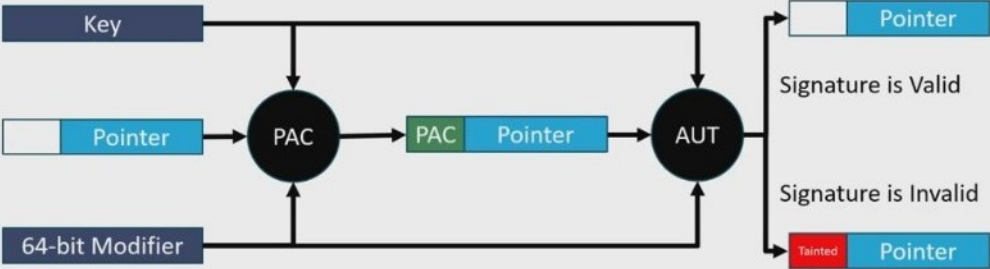
woven
by TOYOTA

Calls and Returns

x86	ARM
CALL • Push return address onto stack • Transfer control to call target	BL and BLR • Put return address into link register • Transfer control to call target
RET • Pop return address off of stack • Transfer control to return address	RET lr • Transfer control to address in register lr



PAC: Signing and Authentication



The diagram illustrates the Pointer Authentication Code (PAC) process. It starts with a 'Key' and a '64-bit Modifier' being inputs to a 'PAC' block. A 'Pointer' is also input to the 'PAC' block. The output of the 'PAC' block is a 'PAC' and a 'Pointer'. This 'PAC' and 'Pointer' are then inputs to an 'AUT' block. The 'Key' and '64-bit Modifier' are also inputs to the 'AUT' block. The output of the 'AUT' block is a decision: 'Signature is Valid' (leading to a 'Pointer') or 'Signature is Invalid' (leading to a 'Tainted' 'Pointer').



Return Address Protection with PAC

```
void f();  
  
int main()  
{  
    f();  
    return 0;  
}  
  
main:  
    pacibsp  
    str    x30, [sp, #-16]!  
    bl     f()  
    mov    w0, wzr  
    ldr    x30, [sp], #16  
    autibsp  
    ret    x30
```



Control Flow Guard vs. Clang CFI

Control Flow Guard	Clang CFI
Coarse-Grained Checks	Finer-Grained Type-Based Checks
Requires Toolchain/OS Cooperation	Completely Toolchain-Based
Works with any Build (Even Debug)	Requires Link-Time Optimization
Incrementally Adoptable	Requires Monolithic Adoption
Checking calls to other DLLs is easy	Checking calls to other DSOs is tricky
Works with any language/toolchain	Clang (C and C++)-specific
Coarse-grained checks just work	Fine-grained checks can be finicky



My takeaway

This talk offered an excellent big-picture overview, but it also left me feeling pleasantly overwhelmed - packed with so many new concepts, mechanisms, tools and ideas as it clearly opened far more doors than it could fully cover/explore in one session.

It also made it very concrete how far the industry has gone to **harden C++ execution** without changing the language itself, and how much **modern security is about the overall eco-system**, as it relies on a tight collaboration between compilers, kernels, and hardware.

It also reminded me that as always in security, raising the bar for defenders doesn't end the problem - it shifts it. Each new protection forces attackers to adapt and invent new techniques.

It reinforced the idea that **crashes are often preferable to silent exploitation**, and that **layered defenses** - even imperfect ones - dramatically raise the bar for attackers.

As a C++ developer, it also reminded me that understanding these mechanisms isn't optional anymore: knowing how the code we write code interacts with CFI, [indirect calls](#), and modern CPUs is part of writing responsible, resilient, and production-grade C++.

👉 Allow me a digression here

I would like to take this opportunity to share a personal opinion.

I was recently reminded how damaging loud but poorly informed voices can be. A popular French tech YouTube channel, [Underscore_](#), released [a video](#) titled "*major companies are abandoning C/C++ for Rust*". Unfortunately, the arguments incriminating C++ relied almost entirely on C++98-style concepts, technics and examples: conflating C and C++, and ignoring **decades of language evolution**. By portraying manual [new/delete](#), fragile lifetimes, and unsafe concurrency as the norm, it presents a picture of C++ that simply no longer reflects contemporain practice.

🔗 The problem is not disagreement or criticism *per se*, but the **impact of large-scale vulgarization when it is built on shaky premises**.

Influencers, federal agencies, and politicians content reaches far beyond engineers; it reaches managers, executives, and policy-influenced decision-makers. When those audiences internalize flawed narratives about C++, the resulting decisions can steer entire organizations - and sometimes the industry - based on fear and misconceptions rather than on facts.

💡 **In a nutshell:** should we stop coding in C++ in 2026 ?

- Yes, if you want to **dogmatically** obey OCND, CISA and FBI, and follow trends in general with poor or no critical thinking.
- If you're **pragmatic** on the other end, then **no**: keep learning, teaching, coding, and creating safe, reliable products ! ❤️

Take for instance those two posts [2025 Will Be the Year of Rust-Why Big Tech Is Ditching C++](#) and [Stop Coding In C and C++, Feds Say](#). It uses the same - often, fallacious - arguments, claiming:

Argument	My perspective
Rust is not a drop-in replacement for C++	Agree
C++ Build system (CMake) is a pain	Partially Agree. CMake is a de-facto build system for many languages.
C++ does not fit high-risk	Legacy C++ don't. Contemporain does.
Poor memory safety	Ever heard of <code>{ type value; } ? std::unique_ptr ?</code> etc.
Tech-debt, maintaining cost endanger teams productivity	Not a language-specific thing
Lack of consistent formatting	<code>clang-format</code> and other tools are meant for that
Need to modernize with modern language features	Not a language-specific thing. No need, but one should.
Lack of safe concurrency primitives	STL's concurrency support library , P2300 sender-receiver , etc.

Also:

- C++ syntax is difficult, the error messages are vague, and even writing basic, safe programs can call for thorough understanding of memory management.
- Modern developers find C++ antiquated, difficult, and hostile to newbies

- Rust's syntax is expressive

Let's code

Disclaimer: I am not familiar with **Rust** yet, and my impressions are based on very limited exposure. As such, **my observations may be incomplete or biased**, just as some of the criticisms often directed at C++ are when they rely on outdated assumptions, as mentioned above. Please give me credit for mentioning and being honest about these limitations.

Also, comparing programming languages honestly is inherently difficult: each comes with its own strengths and trade-offs, which calls for nuance rather than absolute judgments.

That said, I Googled "*production-ready Rust snippet*", found [this one](#) in the first results, and played with it on [play.rust-lang.org](#) here then [godbolt](#).

► Details

🔗 Show me the code !

```
macro_rules! hashmap {

    // Handle key-value pairs
    ($($key:expr => $value:expr),+ $(,)? ) => {{
        let mut map = std::collections::HashMap::new();
        $(
            map.insert($key, $value);
        )+
        map
    }};
}

fn main() {
    let config = hashmap! {
        "timeout" => 30,
        "retries" => 3,
        "debug" => 1
    };

    println!("{:?}", config);
}
```

Which one could somehow translate in C++ like (see here on [godbolt](#)):

```
constexpr auto make_hashmap(auto && ... pairs)
{
    return std::unordered_map { pairs... };
}

auto main() -> int {

    auto config = make_hashmap(
```



```

        std::pair{"timeout", 30},
        std::pair{"retries", 3},
        std::pair{"debug", 1}
    );

    std::println("{} ", config);
}

```

Of course one might prefer using function rather than a macro:

```

fn make_hashmap<K, V, I>(pairs: I) -> HashMap<K, V>
where
    K: std::hash::Hash + Eq,
    I: IntoIterator<Item = (K, V)>,
{
    pairs.into_iter().collect()
}

fn main() {
    let config = make_hashmap([
        ("timeout", 30),
        ("retries", 3),
        ("debug", 1)
    ]);

    println!("{:?}", config);
}

```

which equivalent could be

```

template <class... keys, class ... values>
constexpr auto make_hashmap(std::pair<keys, values> && ... pairs)
{
    using K = typename std::common_type_t<keys...>;
    using V = typename std::common_type_t<values...>;

    return std::unordered_map<K, V>{
        std::forward<decltype(pairs)>(pairs)...
    };
}

auto main() -> int {

    const auto config = make_hashmap(
        std::pair{ "timeout", 30 },
        std::pair{ "retries", 3 },
        std::pair{ "debug", 1 },
    );
}

```

```
std::println("{}\n", config);
}
```

to which one might want to add the following overloads to avoid error-bloat

```
constexpr auto make_hashmap(){
    throw std::invalid_argument{ "cannot deduce map type" };
}

template <class... keys, class ... values>
requires (not requires { typename std::common_type_t<keys...>; })
constexpr auto make_hashmap(std::pair<keys, values> && ...){
    static_assert(false, "mismatched types (key)");
}

template <class... keys, class ... values>
requires (not requires { typename std::common_type_t<values...>; })
constexpr auto make_hashmap(std::pair<keys, values> && ...){
    static_assert(false, "mismatched types (value)");
}
```

Or the `if-constexpr` equivalents.

Find the full [C++ example here on godbolt](#) and [Rust example here on play.rust-lang.org](#) or [here on godbolt](#).

Comparing the amount of assembly lines suprised me: **C++** produced **73 lines**, while **Rust** produced about **1000 lines**. Most likely something I do not understand at that point 🤔. I'll make sure to dig that sometime soon.

So what is the takeaway for me on this ?

Rust's syntax does not inherently strike me as simpler or clearer than contemporain C++'s. For example, Rust's macro system-expressions such as `($($key:expr => $value:expr),+ $(,)? ) => { { /* ... */ } }` can feel quite dense on first encounter, almost reminiscent of [regular expressions](#) in terms of readability, approachability and maintainability.

In the C++ examples I considered, there were no hard-to-manage lifetimes, no raw or user-facing pointers, no new/delete, and no explicit ownership wrappers such as `std::unique_ptr`.

Instead, the code relied on value semantics, (N)RVO and [copy elision](#), and features like [Class Template Argument Deduction \(CTAD\)](#) to significantly reduce boilerplate.

That said, **Rust** does excel at producing compiler error diagnostics that are generally more beginner-friendly, whereas C++ diagnostics tend to be more verbose and detailed, which can be both a strength and a challenge **depending on experience level**.

My view is that compilers provide full, verbose diagnostics primarily to avoid bias and misinterpretation. That makes sense.

Still, in well-understood situations, they could offer a more human-friendly first message - clearly marked as a hint - followed by the complete diagnostic. Ideally, this would be configurable by the user (opt-out ?).

This point reminded me of [Sy Brand's talk C++ Compiler Errors for Humans | Pure Virtual C++ 2023](#).

☞ In Rust - as in C++ - developers are offered multiple ways to achieve the same behavior (especially from a business-logic perspective).

Ultimately, the real distinction lies not (only) in some absolute notion of capability or safety, but more in how each language is taught, positioned, and applied.

Choosing a programming language should remain a **pragmatic decision driven by project constraints**, domain requirements, and long-term maintainability - **not by trends nor ideological preferences**.

Rust is often perceived as easier to learn in part because it is a **younger language**, taught with a clean slate and modern practices from day one.

C++, **however, has evolved** just as significantly over the past decades; when taught starting from contemporary C++ - with value semantics, strong types, RAII, and modern tooling - **it feels like an entirely different language** than the one many still associate with outdated techniques.

💡 C++ super-power is to always adapt & evolve, to meets the industry needs.

As such, a key responsibility lies with educators and experienced developers: to teach modern/contemporain C++, not legacy idioms. When approached this way, learning C++ can be just as accessible and rewarding for beginners, while still offering the **unmatched level of expressiveness, control and performance** that defines its role in the industry.

Conclusion

This conference reinforced something I deeply believe since 2011: **C++ is very much alive, evolving**, and steadily addressing the real concerns of our industry: **safety, performance, and maintainability**, but also education.

Many of the talks converged on the same idea: the language itself is not the problem. Mindsets, outdated practices, misinformation, and fear-driven narratives too often are.

We currently live in an era dominated by trends, buzzwords, and speculative claims, where **loud voices and simplified narratives can overshadow nuance and facts**.

In that context, the C++ community feels to me like a light in the darkness. It is a community that values **rigor over hype, pragmatism over dogmas**, and **long-term thinking over short-term trends**.

As opposed to what many uninformed narratives suggest, C++ has not stagnated or blindly chased novelty. It has evolved by adding meaningful features that extend what developers can express thus achieve, while continuously improving safety, clarity/expressiveness, and accessibility - and, importantly, by simplifying many aspects of everyday development rather than making them more complex.

No, modern C++ does not require developers to scatter raw pointers (*) everywhere, turning code into what C++98 once (and still too often) resembled: a syntactic xmas tree (★ -> 🎄).

No, expressing intent no longer means writing tons of LoCs with deeply nested for-while-for-if-else-break-goto control-flow labyrinths, which "stonks" cognitive complexity.

No, developers are not forced nor encouraged into "OOP" inheritance-heavy, vtable-driven designs at the user level.

These techniques still exist when they are the right tool - but they are no longer the default, nor the necessity. More importantly, delivering value in C++ no longer requires excessive amounts of code.

🔗 Contemporain C++ enables developers to translate ideas directly into expressive, maintainable, and safe abstractions: code that is concise, robust, and adequate for production.

What also stood out is the community's willingness to always **self-criticize, improve, and teach**. That culture of responsibility - combined with a passion for sharing knowledge - is what keeps C++ relevant and trustworthy.

However, is it worth **opposing reason to ignorance, hype, and comforting falsehoods** ? That is a question I keep coming back to.

Engaging in this fight is not free. It demands emotional energy, time, and resilience. Confronting misinformation and intellectual laziness can be deeply demoralizing, especially when loud voices drown out careful reasoning.

And yet, choosing not to act comes at an even higher cost. Remaining silent would mean accepting ideas I know to be wrong - and that is something I could not reconcile with myself.

We may not all change the world, but we are all responsible for the small part we influence: colleagues, friends, local meetups members, but also and especially junior developers and managers.

Why keep pushing back ? Not to win arguments, but to do our part. To be a small spark in the darkness, even when the night feels long.

When engaging in these debates, there is always the risk of being perceived as someone who's resistant/reluctant to change (remember last year's talk from [Peter Sommerlad](#), "[Collective Amnesia](#)?": "*Pay attention - don't become the grumpy old man ranting*").

It is equally difficult to challenge misleading narratives without unintentionally amplifying the very buzz, hype, or trolling one is trying to counter.

Arguing rarely - if not, never - has an immediate impact, even with the most valuable, sourced arguments. No one ever said "*You convinced me, I changed my mind*": admitting you're wrong takes efforts, humility, and critical thinking.

Even when arguments do not persuade immediately, they still plant seeds: ideas that may take time to grow, but can flourish if given space and care.

🔗 If fear, trends, and buzzwords shift the [Overton window](#) in one direction, then it becomes our collective responsibility to push it back toward **reason and facts**.

As conclusion, taken individually, our efforts may seem insignificant. Together, they matter.

By standing united, grounded in facts, education, and pragmatism, we can steadily push back against disinformation and belief-driven narratives. We oppose nuance and subtlety to oversimplification and dogmatism. Progress may be slow, but reason has a quiet strength - and over time, it endures.

Leaving this conference, I feel both reassured and energized.

Reassured that C++ continues to evolve in the right direction, and **energized** to keep learning, teaching, and pushing back against simplistic narratives.

If there is one takeaway to carry forward, it is this: as we stand together, then education and pragmatism will eventually outlast hype and misinformation - and the C++ community embodies that better than ever.

Thanks for reading, and see you next year at Meeting C++! ❤️

[Guillaume](#).

If you enjoyed this paper:

-  Consider leaving a "*like*"
-  Share this paper with your friends, colleagues, teachers, students.
-  You might be interested in last year's: [Conference - Meeting Cpp 2024 - Trip report](#).

About



I am a C++ and software architecture enthusiast with over a decade of experience learning, practicing, and teaching C++ and software design.

My work typically covers coding, codebases health, software architecture and design, tooling, development practices, and project management considerations that directly affect technical outcomes.

I specialize in technical audits and recovery missions, helping teams stabilize complex or struggling projects, and establish reliable foundations for new ones.

I have a strong interest in code reviews, mentoring, and team coaching, with a focus on enabling developers to grow while helping teams deliver reliable, maintainable, and production-ready software.

I regularly teach contemporary C++ and modern software design principles, emphasizing clarity, safety, expressiveness, and long-term maintainability over legacy or cargo-cult practices.

Over the years, I have acted as a lead developer and technical referent, supported architectural decisions, guided projects through critical phases, and coached teams to reliably ship value into production, while also taking part in recruitment processes to help build efficient and sustainable development teams.

[LinkedIn](#) | [Github](#) | [Publications](#)

Related posts

- [Conference - Meeting Cpp 2024 - Trip report](#)
- [Conference - C++ in Paris Conference 2021 - Trip report](#)
- [Conference - Meeting Cpp 2021 - Trip report](#)